JANMITRA

Civic Issue Reporting & Management System for Nagpur (NMC)

4 Civic Categories | 4 NMC Wards | 4 Departments | 4 User Roles

Team ResolveIT -- B.Tech Final Year

Complete 40-Day Antigravity Prompts Guide

Full sequential prompt repository for all 4 team members across every single day

Zero Skipped Days: All 40 Days x 4 Roles = 160 Daily Prompts

Team Members & Dedicated Colors in This Document:

Anas Shaikh	Backend Core + Infrastructure Lead
Anurag Deshmukh	AI Module + Dashboard Backend Lead
Om Gadekar	Frontend Lead (Officer & Worker Portals)
Faizan Rayeen	Citizen Portal & Notifications Lead

Complete 40-Day Project Phase Architecture Roadmap		
Phase 0	Days 01--03	Project Setup, Docker Compose, Architecture Scaffolding
Phase 1	Days 04--10	Foundation: Models, Auth JWT, Leaflet GPS, Notifications v1
Phase 2	Days 11--20	Core Workflow: Citizen Submit, YOLO Pipeline, Worker Tasks
Phase 3	Days 21--28	Department Dashboard: Analytics, Heatmap, Worker Roster
Phase 4	Days 29--36	System Hardening, Test Suite, Docker Production, Cloud Deploy
Phase 5	Days 37--40	Demo Prep, Presentation Slides, Code Cleanup & Viva Release

How to Use This Antigravity Master Prompts Guide

This document contains exact Antigravity prompt instructions for all 4 team members for every single development day (Day 01 to Day 40). Every day block specifies: (1) Key tasks to accomplish, and (2) The exact formatted prompt to copy-paste into Antigravity IDE. All 4 team members work in parallel without merge conflicts, guided by their respective color codes and domain ownership.

Phase 0 -- Project Setup & Environment

Days 01 -- 03

DAY 01 Anas Shaikh

Backend Lead

KEY TASKS:

- Create GitHub repo and invite all 4 team members",
- Scaffold Django project: django-admin startproject resolveIT",
- Create 4 apps: accounts, complaints, dashboard, notifications",
- Set up settings/base.py, local.py, production.py",
- Create docker-compose.yml with: postgres (postgis image), redis, backend, celery, frontend",
- Write Dockerfile for backend and frontend",
- Push to GitHub develop branch",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA, a civic issue reporting system for Nagpur Municipal Corporation.
Tech stack: Django 4.2, DRF, SimpleJWT, PostgreSQL+PostGIS, Celery+Redis, React+Tailwind.

Task: Scaffold the complete Django project structure.

1. Create Django project: resolveIT
2. Create 4 apps: accounts, complaints, dashboard, notifications
3. Set up 3 settings files:
   - settings/base.py: INSTALLED_APPS (include django.contrib.gis), DRF config, JWT config, Celery config
   - settings/local.py: DEBUG=True, PostGIS Docker DB settings
   - settings/production.py: DEBUG=False, reads from env vars
4. Create docker-compose.yml with these services:
   - postgres: image=postgis/postgis:15-3.3, port 5432
   - redis: image=redis:7-alpine, port 6379
   - backend: Python 3.11, runs Django, port 8000
   - celery: same image as backend, runs celery worker
   - celery-beat: runs celery beat
   - frontend: Node 18, Vite React, port 5173
5. Create Dockerfile for backend (Python 3.11) and frontend (Node 18)
6. Create requirements.txt with all packages
7. Create .env.example with all required variables

Generate all files completely. No placeholders.
```

DAY 01 Anurag Deshmukh

AI + Dashboard

KEY TASKS:

- Set up Python environment for AI module",
- Install: pillow, opencv-python-headless, numpy, transformers, torch, ultralytics",
- Create ai_module/ folder structure inside complaints app",
- Test that all AI packages import correctly",
- Download HuggingFace model: umm-maybe/AI-image-detector",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA, a civic issue reporting system.
I need to set up the AI module for fake image detection and YOLO validation.

Task: Set up the AI module structure inside the complaints Django app.

Create this folder structure:
complaints/
```

```
  ai_module/
    __init__.py
    exif_checker.py      # Layer 1 Step 1: EXIF metadata check
    ela_checker.py       # Layer 1 Step 2: Error Level Analysis
    fake_detector.py     # Layer 1 Step 3: HuggingFace AI fake detector
    yolo_validator.py    # Layer 2: YOLOv8 relevance check
    duplicate_detector.py # PostGIS 100m duplicate check
    priority_engine.py   # Weighted priority score
    models/              # Folder for .pt YOLO model file
    __init__.py

For each file: write a complete skeleton class with method signatures,
docstrings explaining what each method does, and placeholder return values.

Also write a test script: test_imports.py that imports all modules and
prints success/failure for each import. The script should not crash if
a model file is missing (just print a warning).

Use these exact package versions in requirements:
pillow==10.4.0, opencv-python-headless==4.10.0, numpy==1.26.4,
transformers==4.44.0, torch==2.3.0, ultralytics==8.2.0
```

DAY 01 Om Gadekar Frontend Lead

KEY TASKS:

- Set up React + Vite + Tailwind CSS project",
- Configure tailwind.config.js with JANMITRA colour tokens",
- Set up React Router v6 with placeholder pages",
- Create AuthContext with React Context API",
- Set up Axios client with JWT interceptor",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA, a civic issue reporting system.
I need to scaffold the React frontend project.

Task: Set up the complete React + Vite + Tailwind project.

1. Project setup: npm create vite@latest frontend -- --template react
2. Install: tailwindcss, react-router-dom, axios, leaflet, react-leaflet, recharts, firebase

3. tailwind.config.js: Add these custom colours:
   primary: #1A56DB, dark: #0F3460, success: #0E9F6E,
   warning: #D97706, danger: #E02424, purple: #7C3AED

4. Create folder structure:
   src/
     context/AuthContext.jsx    -- Context: token, user, role, login(), logout()
     api/client.js              -- Axios instance, baseURL from env, JWT interceptor
     components/                -- Empty folder (components come later)
     pages/
       auth/LoginPage.jsx       -- Placeholder login page
       citizen/Home.jsx         -- Placeholder
       worker/TaskList.jsx      -- Placeholder
       officer/Dashboard.jsx    -- Placeholder
       admin/AdminPanel.jsx     -- Placeholder
     App.jsx                    -- Router with ProtectedRoute HOC

5. AuthContext.jsx: Complete implementation with:
   - token, user, role stored in state + localStorage
   - login(token, userData) and logout() functions
   - isAuthenticated computed value
```

```
6. App.jsx: Complete routing with ProtectedRoute that checks role.
   Routes: /login, /citizen/*, /worker/*, /officer/*, /admin/*

Generate all files completely.
```

DAY 01 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Create Firebase project at console.firebase.google.com",
- Enable Firebase Cloud Messaging (FCM)",
- Get web app configuration (apiKey, projectId, etc.)",
- Set up firebase.js in React project",
- Create firebase-messaging-sw.js service worker",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA, a civic issue reporting system.
I need to set up Firebase Cloud Messaging (FCM) for push notifications.

Task: Set up Firebase in the React frontend for push notifications.

1. Write complete firebase.js setup file:
   - Import: initializeApp, getMessaging, getToken, onMessage
   - Export: app, messaging, requestFCMPermission(), onForegroundMessage()
   - requestFCMPermission(): requests notification permission, gets FCM token,
     calls POST /api/v1/auth/fcm-token/ to save token to backend, returns token
   - onForegroundMessage(callback): registers foreground message handler

2. Write complete firebase-messaging-sw.js service worker:
   - importScripts Firebase compat scripts
   - Initialize Firebase with config
   - Handle background messages: show notification with title, body, icon

3. Write complete FirebaseProvider.jsx:
   - Wraps app, calls requestFCMPermission on login
   - Registers onForegroundMessage to show toast notification
   - Shows a simple toast component when notification arrives

4. Write NotificationBell.jsx component:
   - Bell icon with unread count badge
   - Dropdown showing last 10 notifications
   - Mark all as read button
   - Each notification: title, body, time ago, complaint ID link

Use firebase SDK v10. Show placeholder values for Firebase config
(explain which values to replace from Firebase console).
Generate all files completely.
```

DAY 02 Anas Shaikh — Backend Lead

KEY TASKS:

- Create Custom User model in accounts/models.py",
- Fields: id(UUID), full_name, mobile (login), role, ward, department, designation, employee_id, status, fcm_token",
- Set AUTH_USER_MODEL = 'accounts.User' in settings",
- Run makemigrations and migrate",
- Create Registration and Login serializers",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Continue building the accounts app.

Task: Create the complete Custom User model and authentication serializers.

1. accounts/models.py -- Complete CustomUser model:
   - id: UUIDField(primary_key=True, default=uuid.uuid4)
```

```
   - full_name: CharField(max_length=100)
   - mobile: CharField(max_length=15, unique=True) -- this is the login field (USERNAME_FIELD)
   - email: EmailField(blank=True)
   - role: CharField choices=['citizen','worker','dept_officer','admin'], default='citizen'
   - ward: ForeignKey('complaints.Ward', null=True, blank=True, on_delete=SET_NULL)
   - department: ForeignKey('complaints.Department', null=True, blank=True, on_delete=SET_NULL)
   - designation: CharField(max_length=100, blank=True) -- for workers
   - employee_id: CharField(max_length=50, blank=True, unique=True with null=True)
   - status: CharField choices=['active','inactive','on_leave'], default='active'
   - fcm_token: TextField(blank=True)
   - is_active: BooleanField(default=True)
   - is_staff: BooleanField(default=False) -- needed for admin panel
   - registered_at: DateTimeField(auto_now_add=True)
   - USERNAME_FIELD = 'mobile', REQUIRED_FIELDS = ['full_name']

2. accounts/managers.py -- CustomUserManager:
   - create_user(mobile, full_name, password, **extra)
   - create_superuser(mobile, full_name, password, **extra)

3. accounts/serializers.py:
   - CitizenRegisterSerializer: name, mobile, ward (1-4 choice), password
   - LoginSerializer: mobile, password
   - UserProfileSerializer: all readable fields (no password)
   - WorkerCreateSerializer (admin only): all worker fields

4. accounts/permissions.py:
   - IsCitizen, IsWorker, IsDeptOfficer, IsAdmin classes
   - Each checks request.user.role == 'X' and request.user.is_authenticated

5. accounts/views.py: RegisterView, LoginView (returns JWT), TokenRefreshView wrapper
6. accounts/urls.py: /auth/register/, /auth/login/, /auth/refresh/, /auth/fcm-token/

Generate all files completely. Include all imports.
```

DAY 02 Anurag Deshmukh AI + Dashboard

KEY TASKS:

- Complete exif_checker.py: EXIF metadata Layer 1 check",
- Complete ela_checker.py: Error Level Analysis Layer 1 check",
- Write unit tests for both checkers with sample test images",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Write the AI fake detection module -- Layer 1 Steps 1 and 2.

Task: Complete the EXIF checker and ELA checker.

1. complaints/ai_module/exif_checker.py:
   Complete EXIFChecker class with check(image_path) method:
   - Opens image with Pillow
   - Gets EXIF data using image._getexif()
   - Checks for presence of these EXIF tags:
     * GPS data (tag 34853)
     * Camera Model (tag 272)
     * DateTime (tag 306)
   - Returns: {'real': True/False, 'reason': 'string explaining result', 'exif_found': [list of tags found]}
   - Rule: If 0 tags found: real=False (AI generated or screenshot)
   - Rule: If 1+ tags found: real=True
   - Handle exceptions (image without EXIF support)

2. complaints/ai_module/ela_checker.py:
   Complete ELAChecker class with check(image_path) method:
   - Save image at 90% quality to temp file
```

```
   - Compare original vs recompressed using cv2.absdiff()
   - Calculate mean pixel difference with numpy
   - Returns: {'real': True/False, 'reason': str, 'ela_score': float}
   - Rule: If mean_diff < 5.0: real=False (too uniform = AI generated)
   - Rule: If mean_diff >= 5.0: real=True
   - Clean up temp file in finally block

3. Write test_layer1.py:
   - Test with a sample real image (create a simple test image with Pillow that has EXIF-like properties)
   - Test with a plain image (no EXIF)
   - Test with a solid colour image (will have low ELA score)
   - Print clear PASS/FAIL results

Generate complete files with all imports and error handling.
```

DAY 02 Om Gadekar Frontend Lead

KEY TASKS:

- Create shared UI components: StatusBadge, PriorityBadge, CategoryBadge, WardBadge",
- Create KPICard component for dashboard metrics",
- Create LoadingSpinner and ErrorMessage components",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Create the shared React UI components.

Task: Build all shared/reusable components used across all 4 portals.

1. StatusBadge.jsx -- coloured pill showing complaint status:
   Status colours:
   - Submitted: grey
   - Validated: blue
   - Routed: cyan
   - Assigned: indigo
   - In-Progress: amber (warning)
   - Work Done: purple
   - Closed: green (success)
   - Rejected / Invalid: red (danger)
   Props: status (string)

2. PriorityBadge.jsx -- coloured pill for priority:
   - Critical: red with pulsing animation
   - High: orange
   - Medium: amber
   - Low: grey
   Props: level (string)

3. CategoryBadge.jsx -- icon + colour for each of 4 categories:
   - Roads & Infrastructure: road icon, amber colour
   - Garbage & Waste Management: trash icon, green colour
   - Water & Drainage: droplet icon, blue colour
   - Street Lighting & Electrical: lightbulb icon, yellow colour
   Props: category (string)

4. WardBadge.jsx -- simple ward tag:
   Shows 'Ward 1', 'Ward 2', 'Ward 3', 'Ward 4'
   Small pill with dark blue background.
   Props: ward (number or string)

5. KPICard.jsx -- stat card for dashboard:
   Props: title, value, subtitle, colour ('blue'|'green'|'amber'|'red'|'purple')
   Shows: coloured left border, big number, title, optional subtitle
```

```
6. LoadingSpinner.jsx -- centred spinner
7. ErrorMessage.jsx -- red error box with message prop

Use Tailwind CSS for all styling. Use emoji as icons (no icon library needed).
Generate all components completely.
```

DAY 02 Faizan Rayeen Citizen + Notifications

KEY TASKS:

- Build Login/Register page UI",
- Mobile-first design with form validation",
- Ward selection dropdown (Ward 1-4) in register form",
- Connect to auth API endpoints",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Login and Register page.

Task: Create a complete, professional Login/Register page for all user roles.

LoginPage.jsx requirements:
1. Clean card layout centred on page (mobile-first, max-w-md mx-auto)
2. JANMITRA logo/title at top with tagline
3. Two tabs: 'Login' and 'Register' (toggle between forms)
4. Login form fields:
   - Mobile Number (required, numeric)
   - Password (required, with show/hide toggle)
   - Submit button (loading state during API call)
5. Register form fields:
   - Full Name (required)
   - Mobile Number (required)
   - Ward: dropdown with options [Ward 1, Ward 2, Ward 3, Ward 4]
   - Password (required, min 8 chars)
   - Confirm Password (must match)
   - Submit button

6. API integration:
   - Login: POST /api/v1/auth/login/ -> store token in AuthContext
   - Register: POST /api/v1/auth/register/ -> then auto-login
   - After login: redirect based on role:
     * citizen -> /citizen
     * worker -> /worker
     * dept_officer -> /officer
     * admin -> /admin

7. Error handling: show API error messages below the form
8. Form validation: show red border + error text per field

Design: Tailwind CSS, dark blue header card with JANMITRA in white,
white form card below. Looks professional on mobile.

Generate complete LoginPage.jsx with all logic.
```

DAY 03 Anas Shaikh Backend Lead

KEY TASKS:

- Create static master models: Category (4), Ward (4), Department (4) in complaints/models.py",
- Create management command: seed_demo.py to populate all master data",
- Create Complaint model with all fields",
- Create ComplaintImage and StatusHistory models",
- Run migrations",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Create all the complaint-related models.
```

```
Task: Create all models in complaints/models.py

1. Department model (4 entries):
   - id: SmallIntegerField primary key (1-4)
   - name: CharField (full name)
   - short_name: CharField (PWD, SWM, PHE, ELEC)
   Seed data: 1=Public Works Dept(PWD), 2=Solid Waste Mgmt(SWM),
              3=Public Health Engineering / Water Works(PHE), 4=Electrical Dept(ELEC)

2. Ward model (4 entries):
   - id: SmallIntegerField primary key (1-4)
   - name: CharField (Ward 1, Ward 2, Ward 3, Ward 4)

3. Category model (4 entries):
   - id: SmallIntegerField primary key (1-4)
   - name: CharField
   - department: ForeignKey -> Department
   Seed data:
   1=Roads & Infrastructure -> PWD (1)
   2=Garbage & Waste Management -> SWM (2)
   3=Water & Drainage -> PHE (3)
   4=Street Lighting & Electrical -> Electrical (4)

4. Complaint model:
   - id: UUIDField primary key
   - citizen: FK -> User
   - category: FK -> Category
   - department: FK -> Department (auto-set from category)
   - ward: FK -> Ward (citizen selects)
   - description: TextField
   - location: PointField(geography=True, null=True) -- PostGIS GPS
   - status: CharField choices=[all statuses], default='Submitted'
   - priority_score: FloatField(default=0)
   - priority_level: CharField choices=[Low,Medium,High,Critical], default='Low'
   - fake_detected: BooleanField(default=False)
   - fake_reason: TextField(blank=True)
   - ai_confidence: FloatField(null=True)
   - is_duplicate: BooleanField(default=False)
   - parent_complaint: FK(self, null=True, blank=True)
   - assigned_worker: FK -> User (null=True, related_name='assigned_complaints')
   - ward_override: BooleanField(default=False)
   - override_reason: TextField(blank=True)
   - citizen_rating: SmallIntegerField(null=True, validators=[1-5])
   - submitted_at: DateTimeField(auto_now_add=True)
   - resolved_at: DateTimeField(null=True)

5. ComplaintImage model:
   - complaint: FK -> Complaint
   - image: ImageField
   - image_type: CharField choices=['complaint','proof']
   - uploaded_at: DateTimeField(auto_now_add=True)

6. StatusHistory model:
   - complaint: FK -> Complaint
   - old_status, new_status: CharField
   - changed_by: FK -> User
   - note: TextField(blank=True)
   - changed_at: DateTimeField(auto_now_add=True)

7. Management command: management/commands/seed_demo.py
   Creates: 4 depts, 4 wards, 4 categories, 1 admin user,
```

```
   4 dept officers (one per dept), 16 workers (one per dept+ward combo),
   4 demo citizens (one per ward)

Generate all files completely with migrations.
```

DAY 03 Anurag Deshmukh AI + Dashboard

KEY TASKS:

- Complete fake_detector.py: HuggingFace AI fake detector",
- Complete yolo_validator.py: YOLOv8 relevance check",
- Write the main process_complaint_ai Celery task in complaints/tasks.py",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete the AI module and main Celery task.

Task: Write fake_detector.py, yolo_validator.py, and the main AI Celery task.

1. complaints/ai_module/fake_detector.py:
   Complete FakeDetector class:
   - Load model ONCE at module level (lazy load on first use):
     from transformers import pipeline
     detector = pipeline('image-classification', model='umm-maybe/AI-image-detector')
   - check(image_path) method:
     * Runs detector(image_path)
     * Returns {'real': True/False, 'reason': str, 'confidence': float}
     * Rule: if label == 'artificial' and score >= 0.70 -> real=False
     * Handle: model not found (file missing), image read error

2. complaints/ai_module/yolo_validator.py:
   Complete YOLOValidator class:
   - Load YOLO model from ai_module/models/janmitra.pt on init
   - Fallback: if janmitra.pt missing, use yolov8n.pt (default COCO)
   - validate(image_path_list) method:
     * Runs YOLO on each image
     * Returns max confidence score across all images
     * Returns {'confidence': float, 'objects_found': [list of detected objects]}
     * Rule: confidence >= 0.5 means image is relevant

3. complaints/tasks.py -- Complete process_complaint_ai Celery task:
   @shared_task(bind=True, max_retries=3)
   def process_complaint_ai(self, complaint_id):
     Step 1: Get complaint and its complaint-type images
     Step 2: For each image, run Layer 1 checks in order:
       a. EXIFChecker().check(path)  -- if real=False: reject
       b. ELAChecker().check(path)   -- if real=False: reject
       c. FakeDetector().check(path) -- if real=False: reject
       If any check fails: set complaint.status='Rejected -- Fake Image',
       complaint.fake_detected=True, complaint.fake_reason=reason,
       save, send FCM notification, return
     Step 3: Run YOLOValidator().validate([all image paths])
     Step 4: If confidence >= 0.5: status='Validated'
             Else: status='Pending Validation'
     Step 5: DuplicateDetector().detect(complaint)
     Step 6: PriorityEngine().calculate(complaint)
     Step 7: If Validated: call route_complaint.delay(complaint_id)
     Step 8: Send FCM notification to citizen

   Also write: route_complaint(complaint_id) -- sets status=Routed,
   sends notification to dept officer

Generate all files completely.
```

DAY 03 Om Gadekar Frontend Lead

KEY TASKS:

- Create LeafletMap component: GPS pin display + draggable pin for submission",
- Create ImageUploader component with fake detection spinner UI",
- Test that Leaflet loads correctly with OpenStreetMap tiles",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Create the map and image upload components.

Task: Build LeafletMap and ImageUploader components.

1. LeafletMap.jsx -- Reusable map component:
   Two modes:
   Mode A (picker mode): for complaint submission
   - Shows map centred on user's GPS location (auto-detect with browser geolocation API)
   - Draggable blue marker: citizen adjusts exact pin position
   - Text below map: shows detected address (reverse geocode using Nominatim free API)
   - Props: onLocationChange(lat, lng) callback

   Mode B (display mode): for viewing complaint location
   - Shows fixed red marker at given coordinates
   - No dragging
   - 'Open in Google Maps' link button below
   - Props: lat, lng, readonly=true

   Both modes: OpenStreetMap tiles (no API key needed)
   Tile URL: https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png

2. ImageUploader.jsx -- Multi-photo upload with validation UI:
   - Accepts 1 to 3 photos
   - For each photo after selection:
     * Show thumbnail preview
     * Show 'Verifying photo...' spinner overlay (while EXIF check runs on backend)
     * On verification done: remove spinner, show green checkmark
     * On fake detected: remove photo, show red alert: 'This image cannot be verified.
       Please upload a real photo taken by your phone camera.'
   - Upload button: disabled until at least 1 photo verified
   - Shows photo count: '2/3 photos added'
   - Props: onChange(files), maxPhotos=3

   EXIF check: call POST /api/v1/complaints/verify-image/ (non-auth) with image file
   Returns: {real: true/false, reason: str}

Generate both components completely with Tailwind CSS styling.
```

DAY 03 Faizan Rayeen Citizen + Notifications

KEY TASKS:

- Build Citizen Home page",
- Implement quick stats: My Open + My Resolved complaint counts",
- Recent 3 complaints list with StatusBadge and CategoryBadge",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Citizen Home page.

Task: Create the complete Citizen Home page (citizen/Home.jsx).

Citizen Home page layout (mobile-first):

1. Top greeting bar: 'Welcome back, [Name]!' + ward badge (e.g. 'Ward 2')

2. Large 'Report New Issue' button:
```

```
   - Full width, Primary Blue (#1A56DB)
   - Icon: + symbol
   - Navigates to /citizen/submit
   - Slight shadow + hover scale animation

3. Quick stats row (2 cards side by side):
   - 'My Open Issues': count in amber, link to /citizen/complaints?status=open
   - 'Resolved': count in green, link to /citizen/complaints?status=resolved
   - Fetch from: GET /api/v1/complaints/?my=true&summary=true

4. Notification bell at top right:
   - Import and use NotificationBell component
   - Shows unread notification count badge

5. Recent Complaints section heading: 'Recent Activity'
6. List of last 3 complaints (cards):
   Each card shows:
   - CategoryBadge (left), PriorityBadge (right)
   - WardBadge below category
   - Complaint ID (small, grey text)
   - Status with StatusBadge
   - Date submitted (e.g. '3 days ago')
   - Tap card navigates to /citizen/complaint/:id
   - Loading skeleton while fetching

7. 'View All Complaints' button at bottom

8. If no complaints: empty state with camera icon and 'Be the first to report an issue in your ward!'

API calls:
- GET /api/v1/complaints/?limit=3&ordering=-submitted_at
- GET /api/v1/complaints/summary/ (returns open_count, resolved_count)

Generate complete Home.jsx with all API calls using Axios client.
```

Phase 1 -- Foundation -- Auth, Models, Basic APIs

Days 04 -- 10

DAY 04 Anas Shaikh

Backend Lead

KEY TASKS:

- Complete all auth API views and URL routing",
- Test registration, login, token refresh via Postman",
- Create JWT interceptor test: expired token auto-refresh",
- Document all auth endpoints in Postman collection",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete and test the authentication APIs.

Task: Finalise authentication views, test all endpoints.

1. Complete accounts/views.py:
   - CitizenRegisterView: POST /api/v1/auth/register/
     * Creates user with role='citizen', ward from request
     * Returns: {id, name, mobile, role, ward} + JWT tokens
   - LoginView: POST /api/v1/auth/login/
     * Validates mobile + password
     * Returns: {access, refresh, user: {id, name, role, ward, department}}
   - TokenRefreshView: POST /api/v1/auth/refresh/
     * Standard SimpleJWT refresh
   - FCMTokenUpdateView: PATCH /api/v1/auth/fcm-token/
     * Updates user.fcm_token from request body
```

```
2. Add validation:
   - Register: mobile must be exactly 10 digits
   - Register: ward must be 1, 2, 3, or 4
   - Login: return clear error for wrong credentials (not a 500)

3. Complete accounts/urls.py with all routes

4. Write tests: accounts/tests.py
   - test_citizen_registration_success
   - test_registration_duplicate_mobile_fails
   - test_login_success_returns_jwt
   - test_login_wrong_password_fails
   - test_fcm_token_update

Run: python manage.py test accounts
Generate complete views.py, urls.py, and tests.py
```

DAY 04 Anurag Deshmukh AI + Dashboard

KEY TASKS:

- Complete duplicate_detector.py using PostGIS ST_DWithin",
- Complete priority_engine.py with weighted score formula",
- Write unit tests for duplicate detection and priority scoring",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete the duplicate detector and priority engine.

Task: Write duplicate_detector.py and priority_engine.py

1. complaints/ai_module/duplicate_detector.py:
   DuplicateDetector class with detect(complaint) method:
   - Query: find complaints where:
     * category == complaint.category
     * ward == complaint.ward
     * location within 100 metres of complaint.location (PostGIS ST_DWithin)
     * status NOT IN ['Rejected -- Fake Image', 'Invalid']
     * id != complaint.id (exclude self)
   - Django PostGIS query:
     from django.contrib.gis.measure import D
     from django.contrib.gis.geos import GEOSGeometry
     Complaint.objects.filter(
       location__distance_lte=(complaint.location, D(m=100)),
       category=complaint.category,
       ward=complaint.ward
     ).exclude(id=complaint.id)
   - If duplicates found:
     * Set complaint.is_duplicate = True
     * Set complaint.parent_complaint = earliest duplicate found
     * Save
   - Returns: {'is_duplicate': bool, 'parent_id': UUID or None, 'count': int}

2. complaints/ai_module/priority_engine.py:
   PriorityEngine class with calculate(complaint) method:
   - Inputs (gathered from complaint and related data):
     * severity_score (0-100): based on category and description keywords
       Keywords that increase severity:
       'dangerous', 'injury', 'accident', 'blocked completely', 'flooding' -> +30
       'pothole', 'broken', 'not working' -> +15
       'minor', 'small' -> 0
     * safety_score (0-100): Roads=80, Water=70, Electrical=90, Garbage=40
     * public_impact (0-100): number of duplicates * 20, capped at 100
```

```
     * duplicate_count: from is_duplicate flag
     * dept_capacity (0-100): 100 - (active complaints in dept / total capacity * 100)
   - Formula:
     score = severity*0.30 + safety*0.25 + public_impact*0.20 + dup_count*10*0.15 + dept_capacity*0.10
   - Priority level:
     0-30: Low, 31-60: Medium, 61-80: High, 81-100: Critical
   - Sets complaint.priority_score and complaint.priority_level, saves.
   - Returns: {'score': float, 'level': str}

3. Write tests for both classes with example scenarios.
Generate complete files.
```

DAY 04 Om Gadekar Frontend Lead

KEY TASKS:

- Build Complaint Submission Form -- category selection (4 cards)",
- Ward selector (Ward 1-4 pill buttons)",
- Description textarea with character counter",
- Connect ImageUploader and LeafletMap components",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the complaint submission form.

Task: Create the complete Complaint Submission Form (citizen/SubmitForm.jsx).

Full form layout -- all steps on one page (not a stepper, simpler):

1. CATEGORY SELECTION (4 large cards in 2x2 grid):
   Each card: large icon (emoji), category name, tap to select
   Selected card: blue border + blue background + checkmark
   Categories:
   - Roads & Infrastructure (icon: road emoji, amber accent)
   - Garbage & Waste Management (trash emoji, green accent)
   - Water & Drainage (droplet emoji, blue accent)
   - Street Lighting & Electrical (lightbulb emoji, yellow accent)

2. WARD SELECTION (4 pill buttons in a row):
   Ward 1 | Ward 2 | Ward 3 | Ward 4
   Pre-select user's own ward (from AuthContext)
   Selected: blue filled. Unselected: blue outline.

3. DESCRIPTION textarea:
   - Placeholder: 'Describe the issue clearly (minimum 20 characters)'
   - Live character counter: '45 / 20 minimum'
   - Red border if less than 20 chars on submit attempt

4. PHOTOS section:
   - Import and use ImageUploader component (max 3 photos)

5. LOCATION section:
   - Import and use LeafletMap (picker mode)
   - Shows auto-detected GPS + 'Adjust the pin to the exact issue location'

6. SUBMIT BUTTON (sticky at bottom):
   - Disabled if: no category, no description, no verified photo
   - Shows loading spinner during API call
   - On success: navigate to /citizen/complaint/:id with success toast

7. API: POST /api/v1/complaints/ (multipart/form-data)
   Fields: category, ward, description, location_lat, location_lng, images[]

Error handling: show field-level errors from API response.
```

```
Generate complete SubmitForm.jsx.
```

DAY 04 Faizan Rayeen Citizen + Notifications

KEY TASKS:

- Build My Complaints list page with filter tabs",
- Build Complaint Detail page with status timeline stepper",
- Connect both to backend APIs",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the My Complaints list and detail pages.

Task: Create MyComplaints.jsx and ComplaintDetail.jsx

1. citizen/MyComplaints.jsx:
   - Page title: 'My Complaints'
   - Filter tab bar: All | Open | In-Progress | Resolved (updates API query)
   - Complaint cards list (pull from GET /api/v1/complaints/?my=true&status=X):
     Each card:
     - Top row: CategoryBadge (left) + PriorityBadge (right)
     - Second row: WardBadge + complaint date
     - Third row: StatusBadge + Complaint ID (small)
     - Tap card -> /citizen/complaint/:id
   - Pull-to-refresh (just re-fetch on mount)
   - Loading skeleton: 3 placeholder cards while fetching
   - Empty state per tab (e.g. 'No resolved complaints yet')
   - Pagination: 'Load More' button at bottom

2. citizen/ComplaintDetail.jsx:
   - Fetch: GET /api/v1/complaints/:id/
   - Header: Complaint ID, CategoryBadge, WardBadge, PriorityBadge

   STATUS TIMELINE STEPPER (horizontal on desktop, vertical on mobile):
   Steps: Submitted -> Validated -> Assigned -> In-Progress -> Work Done -> Closed
   - Completed steps: solid blue circle + blue line
   - Current step: pulsing blue circle
   - Future steps: grey circle

   - Complaint photos gallery (scrollable thumbnails, tap to enlarge)
   - Description text
   - Location: small Leaflet map (display mode, read-only)

   VERIFICATION SECTION (only shown when status = 'Work Done'):
   - Section heading: 'Has the issue been resolved?'
   - Proof photos gallery (uploaded by worker)
   - 'Yes, It is Fixed' button (green) -> star rating modal (1-5 stars) -> POST /verify/ {accepted:true, rating:X}
   - 'No, Not Fixed Yet' button (red) -> text area for rejection reason -> POST /verify/ {accepted:false, reason:str}

   Generate complete components.
```

DAY 05 Anas Shaikh Backend Lead

KEY TASKS:

- Build Complaint submission API: POST /api/v1/complaints/",
- Build complaint list and detail APIs",
- Auto-set department from category on save",
- Dispatch process_complaint_ai Celery task after save",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the core complaint APIs.

Task: Complete complaints/views.py and complaints/serializers.py for complaint submission.
```

```
1. complaints/serializers.py:
   ComplaintCreateSerializer:
   - Fields: category (1-4), ward (1-4), description (min 20 chars), location_lat, location_lng, images (list of files)
   - Validate: category must be 1, 2, 3, or 4 (raise ValidationError otherwise)
   - Validate: ward must be 1, 2, 3, or 4
   - Validate: at least 1 image file provided
   - create() method:
     * Auto-set department from category.department
     * Create Complaint object with location = Point(lng, lat)
     * Create ComplaintImage objects for each uploaded image
     * Return complaint

   ComplaintListSerializer: id, category_name, ward_name, status, priority_level, submitted_at
   ComplaintDetailSerializer: all fields + images + status_history + assigned worker name

2. complaints/views.py:
   ComplaintViewSet:
   - create(): saves complaint, dispatches process_complaint_ai.delay(complaint.id), returns complaint id + status
   - list(): GET /api/v1/complaints/ -- filters by citizen if role=citizen, all if admin
     Query params: my=true (filter by request.user), status=X, ward=X
   - retrieve(): GET /api/v1/complaints/:id/
     Citizen can only see their own complaints
   - verify(): POST /api/v1/complaints/:id/verify/
     Citizen accepts or rejects resolution
     accepted=True: status=Closed, save rating, set resolved_at=now
     accepted=False: status=Reopened, save reason in status_history

   Auto-dispatch after save: process_complaint_ai.delay(str(complaint.id))

Generate complaints/serializers.py, complaints/views.py, complaints/urls.py completely.
```

DAY 05 Anurag Deshmukh AI + Dashboard

KEY TASKS:

- Test full AI pipeline end-to-end with a sample real image",
- Test full AI pipeline with a downloaded/screenshot image",
- Fix any errors in the pipeline, document thresholds",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Test and debug the complete AI pipeline.

Task: Write a complete standalone test script for the AI pipeline.

Create test_full_pipeline.py (runs outside Django, standalone):

1. Create or use 3 test images:
   - real_photo.jpg: create a test image with random noise pixels (simulates real photo)
   - uniform_image.jpg: create a solid grey image (simulates AI/screenshot)
   - downloaded.jpg: create an all-white image with text drawn on it (simulates downloaded image)

2. Run each through the complete pipeline:
   a. EXIFChecker().check(path)
   b. ELAChecker().check(path)
   c. FakeDetector().check(path) -- catch if model not downloaded, print skip
   d. YOLOValidator().validate([path]) -- catch if model not loaded, print skip

3. Print a formatted results table:
   Image | EXIF result | ELA result | Fake Detect | YOLO | Final Decision

4. Test DuplicateDetector with mocked complaint object (no DB needed):
   - Show the PostGIS query it would run
   - Explain what the result would be
```

```
5. Test PriorityEngine with mocked complaint:
   - Test: Roads category + 'dangerous pothole causing accidents'
   - Expected: Critical level
   - Test: Garbage + 'small pile of garbage near park'
   - Expected: Low-Medium level

Print all results clearly. Generate complete test_full_pipeline.py
```

DAY 05 Om Gadekar Frontend Lead

KEY TASKS:

- Build Worker Task List page",
- Priority-sorted cards with dept+ward filtering (handled by backend)",
- Connect to GET /api/v1/worker/tasks/ API",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Worker Task List page.

Task: Create worker/TaskList.jsx -- the main page workers see after login.

Worker Task List requirements:

1. Page header: Worker name + dept badge + ward badge
   Example: 'Suresh Patil  |  PWD  |  Ward 2'

2. Filter tab bar: 'Active Tasks' | 'Completed'
   Active: status IN [Assigned, In-Progress, Work Done]
   Completed: status = Closed

3. Task cards (fetched from GET /api/v1/worker/tasks/):
   Each card:
   - TOP: PriorityBadge (left, with red pulsing border if Critical) + WardBadge (right)
   - CategoryBadge below
   - Complaint description (first 80 chars with '...')
   - Status with StatusBadge
   - Submitted date: '2 days ago'
   - Small camera icon if photos available
   - Tap -> /worker/task/:id

4. Sorting: Critical first, then High, Medium, Low
   (Backend already returns sorted. Frontend just renders in order.)

5. Empty state per tab:
   Active: checkmark icon + 'No active tasks in your ward. Great work!'
   Completed: 'No completed tasks yet.'

6. Pull-to-refresh: re-fetch on click/tap of refresh button

7. Priority colour indicators:
   - Critical tasks: red left border on card
   - High: orange left border
   - Medium: amber left border
   - Low: grey left border

API: GET /api/v1/worker/tasks/?tab=active (backend filters by worker's dept+ward)

Generate complete TaskList.jsx with Tailwind CSS.
```

DAY 05 Faizan Rayeen Citizen + Notifications

KEY TASKS:

- Set up Firebase Admin SDK in Django notifications app",
- Write send_push() function in notifications/fcm_service.py",
- Write send_push_notification Celery task",
- Wire notifications to complaint submission status",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Set up the backend notification system.

Task: Complete the notifications app for Firebase push notifications.

1. notifications/fcm_service.py:
   - Initialize Firebase Admin SDK using credentials from env var:
     FIREBASE_CREDENTIALS_PATH (path to service account JSON file)
   - send_push(user_id, title, body, data=None):
     * Gets user from DB by user_id
     * Checks user.fcm_token is not empty
     * Creates firebase_admin.messaging.Message:
       token = user.fcm_token
       notification = Notification(title=title, body=body)
       data = data or {} (complaint_id, status, etc.)
     * Calls messaging.send(message)
     * Logs success or failure (don't crash on FCM error)
     * Returns True/False

2. notifications/tasks.py:
   @shared_task
   def send_push_notification(user_id, title, body, data=None):
     Calls fcm_service.send_push(user_id, title, body, data)

3. notifications/models.py:
   Notification model:
   - recipient: FK -> User
   - title, body: CharField
   - complaint: FK -> Complaint (null=True)
   - is_read: BooleanField(default=False)
   - created_at: DateTimeField(auto_now_add=True)

4. notifications/views.py:
   - GET /api/v1/notifications/ -- list unread notifications for request.user
   - PATCH /api/v1/notifications/mark-read/ -- mark all as read

5. Notification triggers (add these calls to existing tasks/views):
   After complaint submitted: send to citizen: 'Complaint Received', 'Your complaint #ID is being reviewed.'
   After validated: send to citizen: 'Complaint Validated', 'Your complaint has been verified and routed.'
   After assigned: send to worker: 'New Task Assigned', 'You have a new complaint to handle in [ward].'
   After work done: send to citizen: 'Issue Resolved?', 'Please verify if the issue has been resolved.'
   After closed: send to citizen: 'Complaint Closed', 'Thank you for your report.'

Generate all files completely with all imports.
```

DAY 06 Anas Shaikh — Backend Lead

KEY TASKS:

- Build worker assignment API: POST /api/v1/complaints/:id/assign/",
- Enforce dept+ward match rule using validate_worker_assignment()",
- Build GET /api/v1/worker/available/?complaint_id=X API",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the worker assignment API.

Task: Write the assignment system in complaints/views.py

1. complaints/validators.py -- validate_worker_assignment():
   def validate_worker_assignment(worker, complaint, override=False):
     errors = []
     if worker.department_id != complaint.department_id:
       errors.append(f'Worker dept {worker.department} does not match complaint dept {complaint.department}')
     if worker.ward_id != complaint.ward_id:
       errors.append(f'Worker ward {worker.ward} does not match complaint ward {complaint.ward}')
     if errors and not override:
       raise ValidationError({'assignment_error': errors})
     return bool(errors)

2. AssignWorkerView -- POST /api/v1/complaints/:id/assign/:
   Permission: IsDeptOfficer | IsAdmin
   Request body: {worker_id: UUID, override: bool (optional), override_reason: str (optional)}
   Logic:
   - Get complaint by id
   - DeptOfficer: verify complaint.department == request.user.department (else 403)
   - Get worker by worker_id
   - If override=True and request.user.role != 'admin': return 403 (only admin can override)
   - Call validate_worker_assignment(worker, complaint, override)
   - If valid: complaint.assigned_worker = worker, status = 'Assigned'
   - If override: complaint.ward_override = True, complaint.override_reason = override_reason
   - Create StatusHistory entry
   - Send FCM to worker
   - Return updated complaint

3. AvailableWorkersView -- GET /api/v1/worker/available/?complaint_id=X:
   Permission: IsDeptOfficer | IsAdmin
   - Gets complaint by complaint_id
   - DeptOfficer: filter workers where dept==complaint.dept AND ward==complaint.ward AND status=active
   - Admin: filter workers where dept==complaint.dept AND ward==complaint.ward (default), or all if ?override=true
   - Returns list: [worker_id, name, ward, designation, active_task_count]

Generate complaints/validators.py and update complaints/views.py.
```

DAY 06 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Build Worker task update API: status changes",
- Build proof photo upload API",
- Add Mark Work Done validation (must have proof photos)",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the worker task management APIs.

Task: Add worker task status update and proof photo upload to complaints/views.py.

1. WorkerTaskListView -- GET /api/v1/worker/tasks/:
   Permission: IsWorker
   - Always filters: department=request.user.department, ward=request.user.ward
   - Also filters: assigned_worker=request.user OR status='Routed' (unassigned in their dept+ward)
```

```
   - Excludes: Closed, Invalid, Rejected statuses
   - Orders by: priority_score DESC (Critical first)
   - Query param: tab=active|completed

2. WorkerStatusUpdateView -- PATCH /api/v1/complaints/:id/status/:
   Permission: IsWorker (and complaint must be assigned to request.user)
   Request: {status: 'In-Progress' or 'Work Done'}
   Allowed transitions:
   - Assigned -> In-Progress: OK
   - In-Progress -> Work Done: ONLY if complaint has at least 1 proof image
     If no proof image: return 400 {'error': 'Upload proof photos before marking as done'}
   Creates StatusHistory entry on each change.
   Sends FCM notification to citizen on Work Done.

3. ProofPhotoUploadView -- POST /api/v1/complaints/:id/proof-upload/:
   Permission: IsWorker (must be assigned worker)
   - Accepts image files (up to 3)
   - Creates ComplaintImage objects with image_type='proof'
   - Returns updated image list

4. Also add: CitizenVerifyView -- POST /api/v1/complaints/:id/verify/:
   Permission: IsCitizen (must be complaint owner)
   Request: {accepted: bool, rating: int (1-5 if accepted), reason: str (if rejected)}
   accepted=True: status=Closed, rating saved, resolved_at=now
   accepted=False: status=Reopened, reason in StatusHistory
   FCM to worker on reopen.

Generate all views completely.
```

DAY 06 Om Gadekar Frontend Lead

KEY TASKS:

- Build Worker Task Detail page",
- Leaflet map showing complaint location (display mode)",
- Proof photo upload UI and Mark Work Done flow",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Worker Task Detail page.

Task: Create complete worker/TaskDetail.jsx

Task Detail page layout:

1. Header bar:
   - Back arrow button
   - PriorityBadge + CategoryBadge + WardBadge in a row
   - Complaint ID (small)

2. Complaint Photos section:
   - Horizontal scrollable gallery of complaint photos
   - Tap photo to enlarge (simple modal)
   - Section title: 'Citizen Photos'

3. Description section:
   - Full complaint description text
   - Submitted: [date] by [anonymous -- don't show citizen name]

4. Location section:
   - Leaflet map (display mode): red pin at complaint GPS coordinates
   - 'Open in Google Maps' link button:
     Opens: https://maps.google.com/maps?q=[lat],[lng]
   - Address text below map (from reverse geocode)
```

```
5. Action section (conditional on status):
   IF status = 'Assigned':
   - 'Accept Task and Start Work' button (green, full width)
   - API: PATCH /api/v1/complaints/:id/status/ {status: 'In-Progress'}

   IF status = 'In-Progress':
   - 'Upload Proof Photos' section:
     * Use ImageUploader component (type='proof', no fake detection needed)
     * Shows already-uploaded proof photos
   - 'Mark as Work Done' button:
     * Disabled (grey) if no proof photos uploaded
     * Enabled (green) after at least 1 proof photo uploaded
     * Shows tooltip on disabled: 'Upload at least 1 proof photo first'
   - API: PATCH /api/v1/complaints/:id/status/ {status: 'Work Done'}

   IF status = 'Work Done':
   - Shows: 'Waiting for citizen verification'
   - Shows submitted proof photos

6. API: GET /api/v1/complaints/:id/ for full complaint detail

Generate complete TaskDetail.jsx with all states.
```

DAY 06 Faizan Rayeen Citizen + Notifications

KEY TASKS:

- Integrate NotificationBell into all portal layouts",
- Handle notification click: navigate to complaint detail",
- Mark notification as read on click",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise the notification UI integration.

Task: Integrate notifications into all 4 portal layouts.

1. Create a shared Layout.jsx (used by all portals):
   - Top navbar: JANMITRA logo/title, Role badge, Notification bell, Logout button
   - Main content area (children)
   - Bottom tab navigation on mobile (citizen and worker portals)

2. CitizenLayout.jsx (extends Layout):
   Bottom tabs: Home | My Complaints | Profile
   Active tab highlighted in blue.

3. WorkerLayout.jsx (extends Layout):
   Bottom tabs: Tasks | Completed | Profile
   Active tab highlighted in blue.

4. OfficerLayout.jsx (extends Layout):
   Top sidebar on desktop, drawer on mobile:
   Items: Dashboard | Pending Validation | Workers | Settings

5. AdminLayout.jsx (extends Layout):
   Top sidebar:
   Items: All Departments | Workers | Reports | Settings
   Department switcher dropdown at top of sidebar.

6. NotificationBell (update from Day 1):
   - Clicking a notification:
     * Mark as read: PATCH /api/v1/notifications/:id/read/
     * Navigate to /citizen/complaint/:complaint_id (if complaint_id exists)
```

```
   - Notification badge: unread count (poll every 60 seconds)
   - Clicking bell: toggle dropdown
   - Click outside: close dropdown (use useEffect + document event listener)

7. Update App.jsx to wrap each portal with its layout.

Generate all layout files and updated App.jsx completely.
```

DAY 07 Anas Shaikh Backend Lead

KEY TASKS:

- Build all 6 dashboard APIs in dashboard/views.py",
- KPI, Ward Stats, Trends, Worker Workload, High Priority, Pending Validation",
- All APIs filtered by department for officers, unrestricted for admin",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build all dashboard backend APIs.

Task: Complete dashboard/views.py with all 6 API endpoints.

Helper: create a get_dept_filtered_qs(request) function:
  If admin: return Complaint.objects.all() (or filtered by ?dept_id param)
  If dept_officer: return Complaint.objects.filter(department=request.user.department)

1. DeptKPIView -- GET /api/v1/dashboard/kpi/:
   Params: dept_id (admin only), ward_id, date_from, date_to, severity
   Returns:
   {
     total: int, pending: int, in_progress: int, resolved: int, rejected: int,
     avg_resolution_days: float,
     severity_distribution: [{level: str, count: int, percentage: float}]
   }
   avg_resolution_days: average of (resolved_at - submitted_at) for closed complaints

2. WardStatsView -- GET /api/v1/dashboard/ward-stats/:
   Returns per-ward breakdown:
   [{ward_id, ward_name, total, pending, in_progress, resolved, rejected}]
   (for each of the 4 wards)

3. TrendsView -- GET /api/v1/dashboard/trends/:
   Params: groupby=day|week|month, date_from, date_to
   Returns: [{date: str, submitted: int, resolved: int}]
   Use Django's TruncDay/TruncWeek/TruncMonth annotate

4. WorkerWorkloadView -- GET /api/v1/dashboard/workers/:
   Params: ward_id (optional filter)
   Returns: [{worker_id, name, ward_name, active_tasks, completed_week, avg_resolution_days, status}]
   For workers in the officer's department (or all for admin)

5. HighPriorityView -- GET /api/v1/dashboard/high-priority/:
   Returns: complaints where priority IN [High, Critical] AND status NOT IN [Closed, Invalid]
   Ordered by: days_open DESC (oldest first)
   Returns: [{id, category_name, ward_name, priority_level, status, days_open, assigned_worker_name}]

6. PendingValidationView -- GET /api/v1/dashboard/pending-validation/:
   Returns: complaints where status='Pending Validation'
   Includes: ai_confidence score, first image thumbnail URL

7. ManualValidateView -- PATCH /api/v1/dashboard/complaints/:id/validate/:
   Request: {action: 'validate'|'reject', reason: str (if reject)}
   validate: status -> Validated, dispatch route_complaint task
   reject: status -> Invalid, create StatusHistory entry
```

```
Generate complete dashboard/views.py, serializers.py, urls.py
```

DAY 07 Anurag Deshmukh AI + Dashboard

KEY TASKS:

- Write auto_close_unverified Celery periodic task",
- Configure Celery Beat schedule for auto-close (every 6 hours)",
- Test Celery tasks: dispatch manually, verify task runs",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Write the auto-close task and Celery Beat config.

Task: Complete Celery periodic tasks and configuration.

1. Add to complaints/tasks.py:
   auto_close_unverified() -- periodic task:
   @shared_task
   def auto_close_unverified():
     - Query: complaints where status='Work Done' AND
       submitted_at < now() - 72 hours (3 days since work done)
       (Use resolved_at or check last status_history entry timestamp)
     - For each: status -> 'Closed', resolved_at = now()
     - Create StatusHistory: 'Auto-closed after 72 hours without citizen response'
     - Send FCM to citizen: 'Complaint Auto-Closed', 'Your complaint has been automatically closed.'
     - Returns count of closed complaints

2. resolveIT/celery.py -- Configure Celery Beat schedule:
   app.conf.beat_schedule = {
     'auto-close-unverified-every-6-hours': {
       'task': 'complaints.tasks.auto_close_unverified',
       'schedule': crontab(hour='*/6'),
     },
   }

3. Write a test for auto_close_unverified:
   - Create a complaint in 'Work Done' status
   - Manually set submitted_at to 4 days ago
   - Call auto_close_unverified() directly
   - Assert complaint.status == 'Closed'
   - Assert complaint.resolved_at is not None

4. Write a Celery health check view (for debugging):
   GET /api/v1/admin/celery-health/
   Permission: IsAdmin
   - Dispatches a simple test task: add.delay(1, 2)
   - Returns: {celery_status: 'ok', result: 3} or error

5. Document Celery setup for team:
   - How to run celery worker: celery -A resolveIT worker --loglevel=info
   - How to run celery beat: celery -A resolveIT beat --loglevel=info
   - On Windows: use Docker (do not use --pool=solo in production)

Generate all task code and configuration.
```

DAY 07 **Om Gadekar** Frontend Lead

KEY TASKS:

- Build Department Officer Dashboard page",
- KPI cards row, Ward bar chart (Recharts), Severity pie chart",
- Dashboard filters: date range, ward, status, severity",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Department Dashboard page.

Task: Create officer/Dashboard.jsx -- the main department dashboard.

Dashboard layout (desktop-first, sidebar layout):

1. Page header bar:
   - Department name: e.g. 'Public Works Department Dashboard'
   - Department colour accent on left border
   - Date range filter (Last 7d | Last 30d | Last 90d | Custom)

2. Filter bar (horizontal, below header):
   Ward: [All | Ward 1 | Ward 2 | Ward 3 | Ward 4]
   Status: [All | Pending | In Progress | Resolved | Rejected]
   Severity: [All | Critical | High | Medium | Low]
   Apply Filters button (blue) -- updates all widgets

3. KPI Cards Row (5 cards):
   KPICard components:
   - Total Complaints (blue)
   - Pending (amber)
   - In Progress (purple)
   - Resolved (green)
   - Rejected (red)
   + Separate card: Avg. Resolution Time (e.g. '3.2 days')
   API: GET /api/v1/dashboard/kpi/ with filter params

4. Charts Row (2 columns):
   Left: Ward-wise Complaint Count (Recharts BarChart):
   - X axis: Ward 1, Ward 2, Ward 3, Ward 4
   - Grouped bars: Pending (amber), In-Progress (purple), Resolved (green)
   - Responsive container
   API: GET /api/v1/dashboard/ward-stats/

   Right: Severity Distribution (Recharts PieChart):
   - Donut chart: Critical (red), High (orange), Medium (amber), Low (grey)
   - Legend below with counts
   API: GET /api/v1/dashboard/kpi/ severity_distribution field

5. Complaint Trend (full width, Recharts LineChart):
   - Toggle: Day | Week | Month
   - Two lines: Submitted (blue), Resolved (green)
   API: GET /api/v1/dashboard/trends/?groupby=day

6. Worker Workload Table (below charts):
   Columns: Name, Ward, Active Tasks, Completed This Week, Avg. Days, Status
   Row click -> opens worker detail modal
   API: GET /api/v1/dashboard/workers/

All data re-fetched when filters change.
Generate complete Dashboard.jsx using Recharts and Tailwind CSS.
```

DAY 07 Faizan Rayeen Citizen + Notifications

KEY TASKS:

- Build Admin Worker Management page",
- List all workers with dept+ward filters",
- Worker registration form (admin/officer creates new workers)",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Worker Management pages.

Task: Create admin/WorkerManagement.jsx and WorkerRegistrationForm.jsx

1. admin/WorkerManagement.jsx:
   - Page title: 'Worker Management'
   - Search bar: search by name or employee ID
   - Filter bar: Department [All|PWD|SWM|PHE|Electrical] + Ward [All|1|2|3|4] + Status [All|Active|Inactive|On Leave]
   - 'Add New Worker' button (top right, blue)
   - Worker table:
     Columns: Employee ID | Name | Dept. | Ward | Designation | Active Tasks | Status | Actions
     Actions: Edit (pencil) | Deactivate/Activate (toggle)
   - Empty state: 'No workers found. Click Add New Worker to start.'
   - Pagination: 20 per page
   API: GET /api/v1/admin/workers/ with filter params

2. WorkerRegistrationForm.jsx (modal or page):
   Fields:
   - Employee ID (required, unique NMC ID)
   - Full Name (required)
   - Mobile Number (required, 10 digits)
   - Email (optional)
   - Department: dropdown [PWD | Solid Waste Management | Public Health Eng. / Water Works | Electrical]
   - Ward: dropdown [Ward 1 | Ward 2 | Ward 3 | Ward 4]
   - Designation: text input (e.g. Field Inspector, Supervisor, Technician)
   - Initial Password: text input (hidden, admin sets it)
   - Confirm Password
   Submit: POST /api/v1/admin/workers/
   Success: close modal, refresh worker list, show success toast

3. Worker Edit modal:
   Same form but pre-filled. Can update: designation, ward, department (admin), status.
   Cannot update: mobile, employee_id.
   Submit: PATCH /api/v1/admin/workers/:id/

4. Dept. Officer view (limited):
   - Shows only workers in their department
   - Can add workers for their department only (dept field pre-set and disabled)
   - Can change ward within their dept only

Generate complete WorkerManagement.jsx and WorkerRegistrationForm.jsx
```

DAY 08 Anas Shaikh Backend Lead

KEY TASKS:

- Build Worker Registration APIs (admin + officer)",
- Build worker list, update, deactivate APIs",
- Add department isolation for dept officer role",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Worker Management backend APIs.

Task: Create accounts/views.py worker management views.

1. WorkerListCreateView -- GET/POST /api/v1/admin/workers/:
```

```
   Permission: IsAdmin | IsDeptOfficer
   GET:
   - Admin: all workers (filterable by dept, ward, status)
   - DeptOfficer: only workers in their department
   - Params: dept_id, ward_id, status, search (name or employee_id)
   - Returns: [{id, employee_id, name, mobile, dept, ward, designation, active_task_count, status}]

   POST (register new worker):
   - Admin: can set any dept + ward
   - DeptOfficer: department auto-set to their dept, cannot override
   - Validation:
     * mobile must be unique
     * employee_id must be unique
     * ward must be 1-4
     * department must be 1-4
   - Creates User with role='worker'
   - Returns 201 with worker data

2. WorkerDetailView -- GET/PATCH /api/v1/admin/workers/:id/:
   Permission: IsAdmin | IsDeptOfficer (own dept only)
   PATCH:
   - Allowed fields: designation, status, email, ward, department
   - DeptOfficer: cannot change department (their dept only)
   - Admin: can change any field
   - mobile and employee_id: read-only (return error if included in request)
   - Log change in admin audit (simple print/log for now)

3. WorkerSearchView -- GET /api/v1/worker/available/:
   (Already built on Day 6 -- verify it's complete and tested)

4. Write tests: accounts/tests/test_workers.py:
   - test_admin_can_register_worker
   - test_officer_can_register_worker_own_dept
   - test_officer_cannot_register_for_other_dept
   - test_duplicate_mobile_fails
   - test_admin_can_change_dept
   - test_officer_cannot_change_dept

Generate accounts/views.py (worker section) and tests.
```

DAY 08 **Anurag Deshmukh** AI + Dashboard

KEY TASKS:

- Connect dashboard KPI API to real complaint data (not mock)",
- Test all 6 dashboard APIs with Postman",
- Fix any data query issues, add proper Django ORM annotations",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise and test dashboard APIs.

Task: Fix and complete all 6 dashboard API data queries.

Review and fix each dashboard view for correctness:

1. DeptKPIView -- verify these counts are accurate:
   - pending: status in ['Routed', 'Validated'] (not yet assigned)
   - in_progress: status in ['Assigned', 'In-Progress']
   - resolved: status == 'Closed'
   - rejected: status in ['Invalid', 'Rejected -- Fake Image']
   - avg_resolution_days: annotate with resolved_at-submitted_at, only for Closed complaints
   - severity_distribution: use values('priority_level').annotate(count=Count('id'))
```

```
2. TrendsView -- fix Django annotation:
   from django.db.models.functions import TruncDay, TruncWeek, TruncMonth
   Use appropriate Trunc based on groupby param.
   Return format: [{date: 'YYYY-MM-DD', submitted: int, resolved: int}]

3. WorkerWorkloadView -- fix active_task_count:
   Annotate users queryset:
                                                                    .annotate(active_tasks=Count('assigned_complaints',
filter=Q(assigned_complaints__status__in=['Assigned','In-Progress'])))
   .annotate(completed_week=Count('assigned_complaints', filter=Q(assigned_complaints__resolved_at__gte=last_week)))

4. HighPriorityView -- add days_open calculation:
   Annotate with: days_open = ExpressionWrapper(Now() - F('submitted_at'), output_field=DurationField())
   Convert to integer days in Python before returning.

5. Write dashboard API tests: dashboard/tests.py
   - Create seed data: 4 complaints across different wards and priorities
   - Test KPI counts are correct
   - Test ward stats sums match total
   - Test trends return correct date groupings

Generate updated dashboard/views.py and tests.
```

DAY 08 Om Gadekar Frontend Lead

KEY TASKS:

- Build High-Priority Complaint list widget on dashboard",
- Build Pending Validation queue page",
- Build Manual Validate/Reject UI for officer",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build remaining officer dashboard components.

Task: Build HighPriorityList and ValidationQueue components.

1. HighPriorityList.jsx -- widget on Dashboard page:
   Table with columns: ID | Category | Ward | Priority | Status | Days Open | Assigned To | Action
   Features:
   - Critical rows: red background tint
   - High rows: orange left border
   - Days Open: red text if > 5 days, amber if 3-5, grey if < 3
   - 'Assigned To': shows worker name or 'Unassigned' in red text
   - 'Action' column: 'View' button -> opens complaint modal
   API: GET /api/v1/dashboard/high-priority/

   ComplaintDetailModal (reusable):
   - Triggered from HighPriorityList and ValidationQueue
   - Shows: complaint photos, description, category, ward, priority, status timeline
   - 'Assign Worker' button (opens AssignWorker sub-modal)

2. officer/ValidationQueue.jsx -- full page:
   - Page title: 'Pending AI Validation'
   - Subtitle: 'These complaints need manual review (AI confidence below 50%)'
   - For each complaint card:
     * Complaint photo (thumbnail, click to enlarge)
     * AI Confidence bar: e.g. '38% confidence' (red bar if < 30%, amber if 30-50%)
     * Category badge, Ward badge, description
     * Two buttons:
       - 'Validate & Route' (green): PATCH /api/v1/dashboard/complaints/:id/validate/ {action:'validate'}
       - 'Reject as Invalid' (red): shows reason textarea -> PATCH with {action:'reject', reason: str}
   - On validate: card disappears from queue, success toast
   - On reject: card disappears, reason logged
```

```
   API: GET /api/v1/dashboard/pending-validation/

3. officer/AssignWorker.jsx -- modal component:
   - Complaint info at top (category, ward, description excerpt)
   - Section: 'Available Workers (Dept + Ward match)'
   - Worker list: radio button per worker, shows name, designation, active task count
   - If no workers: yellow warning 'No workers available for this dept+ward. Use admin override.'
   - Admin Override section (only shown for admin role):
     * Checkbox: 'Override ward rule'
     * Reason textarea (required if checked)
     * Dropdown shows ALL active workers of correct dept
   - 'Assign' button
   API: GET /api/v1/worker/available/?complaint_id=X, POST /api/v1/complaints/:id/assign/

Generate all components completely.
```

DAY 08 Faizan Rayeen Citizen + Notifications

KEY TASKS:

- Build Admin Cross-Department Dashboard",
- 4 dept comparison cards",
- Cross-dept bar chart (Recharts)",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Admin cross-department dashboard.

Task: Create admin/CrossDeptDashboard.jsx

Admin Dashboard layout:

1. Page title: 'JANMITRA Admin Dashboard'
   Subtitle: 'All Departments Overview'

2. Department Switcher (top bar):
   Toggle: [All Departments | PWD | Solid Waste | Water Works | Electrical]
   When specific dept selected: shows same Dashboard.jsx as officer but for that dept
   When 'All Departments': shows cross-dept view below

3. Cross-Dept Summary (4 cards in a row):
   One KPI card per department:
   Each card shows: Dept name (coloured), Total, Pending, Resolved, Avg. Days
   Cards use department colour accents:
   - PWD: blue, SWM: green, PHE: cyan, Electrical: yellow
   API: Call GET /api/v1/dashboard/kpi/?dept_id=1 for each of 4 depts

4. Cross-Dept Bar Chart (Recharts):
   X axis: 4 categories (Roads, Garbage, Water, Street Lighting)
   Bars: Total complaints per category
   API: GET /api/v1/dashboard/kpi/ for all (no dept filter)

5. Ward Heatmap table (simple text heatmap):
   Rows: Ward 1, Ward 2, Ward 3, Ward 4
   Columns: PWD, SWM, PHE, Electrical
   Each cell: complaint count (with colour: red if high, green if low)
   API: 4 calls to GET /api/v1/dashboard/ward-stats/?dept_id=X

6. Quick Links section:
   - 'Register New Worker' -> /admin/workers?action=new
   - 'View All Complaints' -> /admin/complaints
   - 'Pending Validation (all depts)' -> /officer/validation

Generate complete CrossDeptDashboard.jsx with Recharts and Tailwind.
```

DAY 09 Anas Shaikh Backend Lead

KEY TASKS:

- Code review: check all APIs are protected with correct permissions",
- Write API integration tests for complete complaint lifecycle",
- Fix any bugs found in review",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Do a complete security and API review.

Task: Write a comprehensive test that covers the complete complaint lifecycle.

Create complaints/tests/test_lifecycle.py:

Test: test_complete_complaint_lifecycle()
1. Create test users: citizen (ward=1), worker (dept=1, ward=1), officer (dept=1)
2. Citizen submits complaint: category=1 (Roads), ward=1, description, image
3. Assert: complaint created, status=Submitted, dept=PWD (auto-set)
4. Run process_complaint_ai task synchronously (use CELERY_TASK_ALWAYS_EAGER=True in test settings)
5. Assert: complaint status changed (Validated or Pending Validation)
6. Officer assigns worker: worker.dept=PWD, worker.ward=Ward1 (valid assignment)
7. Assert: complaint.assigned_worker = worker, status=Assigned
8. Test invalid assignment: try to assign worker from Ward 2 to Ward 1 complaint
   Assert: raises ValidationError (dept+ward mismatch)
9. Worker marks In-Progress
   Assert: status=In-Progress
10. Worker tries to mark Work Done without proof photo
    Assert: returns 400 error
11. Worker uploads proof photo, marks Work Done
    Assert: status=Work Done
12. Citizen verifies resolution (accepts)
    Assert: status=Closed, rating saved, resolved_at set
13. Test auto_close: create Work Done complaint with old date, run auto_close task
    Assert: status=Closed

Also test RBAC:
- Citizen cannot access /worker/tasks/
- Worker cannot access /dashboard/kpi/
- Officer can only see their own dept in dashboard
Generate the complete test file.
```

DAY 09 Anurag Deshmukh AI + Dashboard

KEY TASKS:

- Begin YOLO model fine-tuning on Google Colab",
- Download civic issue dataset from Roboflow",
- Start training -- 50 epochs on T4 GPU",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Write the Google Colab YOLO training notebook.

Task: Write a complete Google Colab Python script for training YOLOv8 on civic issues.

Create: colab_yolo_training.py (to be run in Google Colab cell by cell)

Cell 1: Install dependencies
!pip install ultralytics roboflow

Cell 2: Download dataset from Roboflow
from roboflow import Roboflow
rf = Roboflow(api_key="YOUR_ROBOFLOW_API_KEY")
project = rf.workspace("civic-issues").project("pothole-detection")
dataset = project.version(1).download("yolov8")
```

```
# Download 3 datasets and merge:
# 1. Pothole Detection (Roboflow)
# 2. Garbage Detection (Roboflow)
# 3. Road Damage Detection

Cell 3: Create merged dataset config (data.yaml):
Write a script that:
- Merges downloaded datasets into one folder: dataset/train/, dataset/val/, dataset/test/
- Creates dataset/data.yaml with:
  path: /content/dataset
  train: train/images
  val: val/images
  nc: 4  (number of classes)
  names: [pothole, garbage, waterlogging, broken_streetlight]

Cell 4: Train YOLOv8
from ultralytics import YOLO
model = YOLO('yolov8n.pt')  # nano = fastest
results = model.train(
  data='/content/dataset/data.yaml',
  epochs=50,
  imgsz=640,
  batch=16,
  device=0,  # GPU
  project='/content/runs',
  name='janmitra_v1'
)

Cell 5: Evaluate and save
# Show results
print(results.results_dict)
# Save best model
import shutil
shutil.copy('/content/runs/janmitra_v1/weights/best.pt', '/content/drive/MyDrive/janmitra_yolo.pt')
print('Model saved to Google Drive')

Cell 6: Quick inference test
model = YOLO('/content/drive/MyDrive/janmitra_yolo.pt')
results = model('/content/test_image.jpg')
results[0].show()

Write the complete notebook code with comments explaining each step.
```

DAY 09 Om Gadekar Frontend Lead

KEY TASKS:

- Integration test: complete citizen complaint submission flow in browser",
- Fix any frontend errors found",
- Ensure ImageUploader fake detection flow works end-to-end",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Write end-to-end test scenarios for the frontend.

Task: Write detailed manual test scripts and fix any identified UI issues.

Create: frontend_test_plan.md

SCENARIO 1: Citizen Submission Flow
1. Open http://localhost:5173
2. Click Register. Fill: Name=Test Citizen, Mobile=9876543210, Ward=Ward 2, Password=test1234
3. Submit. Should redirect to /citizen
4. Click 'Report New Issue'
```

```
5. Select category: 'Roads & Infrastructure' (should highlight with blue border)
6. Ward: Ward 2 (should be pre-selected)
7. Description: type 'Large pothole on main road causing accidents'
8. Upload photo: use a camera photo from phone (should show Verifying... then green tick)
9. Upload a screenshot: should show red error 'cannot be verified'
10. Location: map should auto-detect GPS, pin should be draggable
11. Click Submit: should show loading, then redirect to /citizen/complaint/:id
12. Status should be 'Submitted' then update to 'Validated' (within 30 seconds)
13. Check push notification received in browser

Expected issues to look for:
- Map not loading: check if Leaflet CSS is imported in main.jsx
- CORS error: check django-cors-headers CORS_ALLOWED_ORIGINS includes http://localhost:5173
- JWT not refreshing: check Axios interceptor is attached
- Image upload fails: check MEDIA_ROOT and MEDIA_URL in Django settings

Write the test plan AND fix the top 5 most common setup issues with their solutions.
Also write: src/utils/errorHandler.js that formats Django DRF error responses into user-friendly messages.
```

DAY 09 Faizan Rayeen Citizen + Notifications

KEY TASKS:

- Test all FCM push notification flows",
- Ensure foreground and background notifications both work",
- Test notification bell: click, mark read, navigate to complaint",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Test and fix the complete notification system.

Task: Write notification system tests and a debugging guide.

1. Write: notification_test_guide.md
   Step by step guide to test every notification:

   Test 1: Complaint Submitted notification
   - Submit complaint as citizen
   - Expected: browser notification 'Complaint Received'
   - Check: notification appears in bell dropdown

   Test 2: Complaint Validated notification
   - Run AI pipeline (or manually change status)
   - Expected: notification 'Complaint Validated'

   Test 3: Worker Assigned notification
   - Officer assigns worker
   - Expected: worker browser receives 'New Task Assigned'

   Test 4: Background notification (app closed)
   - Close JANMITRA tab
   - Trigger status change
   - Expected: OS-level notification appears

2. Write: notifications/management/commands/send_test_notification.py
   Management command:
   python manage.py send_test_notification --user-id=<UUID>
   - Sends a test FCM push to the specified user
   - Useful for debugging FCM setup

3. Common FCM issues and fixes:
   - VAPID key missing: how to get from Firebase console
   - Service worker not registered: check public/ folder and HTTPS requirement
   - Notification permission blocked: how to reset in browser settings
```

```
   - FCM token not saved: check if PATCH /auth/fcm-token/ is called after login

4. Update firebase.js to handle:
   - requestFCMPermission returns null if permission denied (show gentle prompt to enable)
   - onForegroundMessage: shows a styled toast with title, body, and 'View' button

Generate all files and guides completely.
```

DAY 10 Anas Shaikh Backend Lead

KEY TASKS:

- Final API review: make sure all 24 endpoints are working",
- Update Postman collection with all endpoints, request bodies, responses",
- Write API documentation README",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise the API documentation.

Task: Write complete API documentation and verify all endpoints.

Create: API_DOCUMENTATION.md

Format each endpoint as:
## [METHOD] /api/v1/endpoint/
Permission: [roles]
Request: {json body or params}
Response 200: {json}
Response 4xx: {error format}
Example curl command

Document ALL 24 endpoints:
1. POST /api/v1/auth/register/
2. POST /api/v1/auth/login/
3. POST /api/v1/auth/refresh/
4. PATCH /api/v1/auth/fcm-token/
5. GET /api/v1/categories/
6. GET /api/v1/wards/
7. POST /api/v1/complaints/
8. GET /api/v1/complaints/
9. GET /api/v1/complaints/:id/
10. PATCH /api/v1/complaints/:id/status/
11. POST /api/v1/complaints/:id/assign/
12. POST /api/v1/complaints/:id/proof-upload/
13. POST /api/v1/complaints/:id/verify/
14. GET /api/v1/worker/tasks/
15. GET /api/v1/worker/available/?complaint_id=X
16. GET /api/v1/dashboard/kpi/
17. GET /api/v1/dashboard/ward-stats/
18. GET /api/v1/dashboard/trends/
19. GET /api/v1/dashboard/workers/
20. GET /api/v1/dashboard/high-priority/
21. GET /api/v1/dashboard/pending-validation/
22. PATCH /api/v1/dashboard/complaints/:id/validate/
23. GET /api/v1/admin/workers/
24. POST /api/v1/admin/workers/
25. PATCH /api/v1/admin/workers/:id/

Also write: a Postman collection JSON file (postman_collection.json) with all 24 requests pre-configured with:
- Variables: base_url, access_token
- Pre-request script to auto-add Authorization header
- Example request bodies for each POST/PATCH
```

DAY 10 Anurag Deshmukh AI + Dashboard

KEY TASKS:

- Download trained YOLO model from Colab",
- Place janmitra_yolo.pt in complaints/ai_module/models/",
- Test yolo_validator.py with the fine-tuned model",
- Document model accuracy results",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise and document the YOLO model.

Task: Write model evaluation script and document AI module performance.

1. Write: evaluate_yolo.py (run locally after downloading model):
   - Load janmitra_yolo.pt model
   - Test on 10 sample images (5 real civic issues, 5 irrelevant images)
   - For each: print detected objects, confidence scores, final decision (relevant/irrelevant)
   - Print accuracy summary

2. Write: AI_MODULE_README.md
   Complete documentation of the AI module:

   Section 1: Overview -- Two-layer validation diagram
   Section 2: Layer 1 -- Fake Detection
     - EXIF check: what it detects, threshold (0 tags = fake)
     - ELA check: what it detects, threshold (mean_diff < 5.0 = fake)
     - HuggingFace: model name, threshold (0.70 confidence)
   Section 3: Layer 2 -- YOLO Validation
     - Model: YOLOv8n fine-tuned on civic dataset
     - Dataset: pothole + garbage + road damage (Roboflow)
     - Classes: pothole, garbage, waterlogging, broken_streetlight
     - Threshold: confidence >= 0.5
   Section 4: Duplicate Detection
     - PostGIS ST_DWithin radius: 100 metres
     - Same category + same ward requirement
   Section 5: Priority Engine
     - Formula and weights
     - Priority level thresholds
   Section 6: Setup Instructions
     - How to download HuggingFace model
     - How to place YOLO model file
     - How to run tests

3. Update fake_detector.py to handle model not found gracefully:
   - If model file not found: log warning, return {'real': True, 'reason': 'model_not_loaded'}
   - This prevents crash if HuggingFace model is not downloaded yet

Generate all files completely.
```

DAY 10 Om Gadekar Frontend Lead

KEY TASKS:

- Complete Admin portal: cross-dept view is working",
- Integration test: officer login, view dashboard, assign worker",
- Fix any chart rendering issues in Recharts",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete and test the officer and admin portals.

Task: Final integration test + fixes for officer and admin portals.

1. Test scenario -- Officer Login Flow:
   - Login as dept_officer (e.g. PWD officer)
```

```
   - Should auto-redirect to /officer/dashboard
   - Dashboard should show ONLY PWD complaints
   - Ward filter works: selecting Ward 2 updates all charts
   - Worker workload table shows PWD workers only
   - 'Assign Worker' works: shows only PWD workers matching complaint ward

2. Test scenario -- Admin Login Flow:
   - Login as admin
   - Redirect to /admin/dashboard
   - Cross-dept view shows 4 department cards
   - Dept switcher: switching to SWM shows SWM data only
   - Worker management: shows all 16 demo workers, filterable

3. Common Recharts issues and fixes:
   - ResponsiveContainer height: must have a fixed parent height
     Add: parent div className='h-64' around ResponsiveContainer
   - CustomTooltip: show formatted numbers
   - Legend alignment: use layout='vertical' for vertical legend

4. Write: src/utils/chartHelpers.js
   Utility functions:
   - formatStatusLabel(status): converts API status string to display label
   - formatPriorityColour(level): returns Tailwind colour class
   - formatDeptName(id): returns full department name
   - formatWardName(id): returns 'Ward X'
   - formatDate(dateStr): formats ISO date to '3 days ago' or '12 Sep 2026'

5. Final frontend checklist:
   - All routes are protected with ProtectedRoute
   - All API errors show user-friendly messages (not raw JSON)
   - Loading states on all data-fetching components
   - No console errors in browser dev tools

Generate chartHelpers.js and a checklist of remaining frontend tasks.
```

DAY 10 Faizan Rayeen Citizen + Notifications

KEY TASKS:

- Final citizen portal polish: forms, error messages, loading states",
- Profile page: citizen can see their ward, update name",
- Help page: how to use JANMITRA (simple FAQ)",

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the citizen profile and help pages.

Task: Build citizen/Profile.jsx and citizen/Help.jsx

1. citizen/Profile.jsx:
   - Shows: Name, Mobile (masked: ***3210), Ward, Account Status
   - Edit name button: inline edit field + save
   - Change Password: old password + new password + confirm
   - API: GET /api/v1/auth/profile/, PATCH /api/v1/auth/profile/
   - My Stats section:
     * Total complaints submitted
     * Resolved count
     * Pending count
     * Average rating given (if rated any)
   - 'Logout' button (red, bottom of page): clears AuthContext + localStorage

2. citizen/Help.jsx:
   - Title: 'How to Use JANMITRA'
   - FAQ accordion cards (click to expand):
```

```
    Q: 'How do I report an issue?'
    A: 'Tap Report New Issue, select the issue type...'
    Q: 'Why was my photo rejected?'
    A: 'JANMITRA uses AI to verify that photos are real camera photos...'
    Q: 'How do I track my complaint?'
    A: 'Go to My Complaints and tap on any complaint...'
    Q: 'What happens after I submit?'
    A: 'Your complaint is reviewed by AI, routed to the correct department...'
    Q: 'How long does resolution take?'
    A: 'Depends on issue type and priority. Critical issues target 24-48 hours...'
    Q: 'Can I reopen a complaint?'
    A: 'Yes, if the issue was not properly fixed, tap Reject on the Work Done notification.'

3. Add bottom navigation to CitizenLayout:
   Tabs: Home | My Complaints | Help | Profile
   Active tab: blue icon + blue text

Generate all files completely with Tailwind CSS.
```

Phase 2 -- Core Workflow -- AI, Assignment, Worker, Citizen

Days 11 -- 20

DAY 11 Anas Shaikh

Backend Lead

KEY TASKS:

- Create complaints/routing.py with department routing table
- Implement auto_route_complaint Celery task
- Test routing across all 4 categories to 4 NMC departments

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the automated complaint routing engine.
Task: Create complaints/routing.py and routing Celery task.
1. Department Mapping Table:
   - Roads & Infrastructure -> PWD (id=1)
   - Garbage & Waste -> Solid Waste Management (id=2)
   - Water & Drainage -> Public Health Engineering (id=3)
   - Street Lighting -> Electrical Department (id=4)
2. routing.py functions:
   - get_department_for_category(category_id)
   - route_complaint(complaint_id): updates complaint.department, status='Routed', logs to StatusHistory
3. Celery Task in complaints/tasks.py:
   - @shared_task route_complaint_task(complaint_id)
   - Fires automatically after AI validation succeeds
Generate complete routing.py and update tasks.py.
```

DAY 11 Anurag Deshmukh

AI + Dashboard

KEY TASKS:

- Assemble validate_complaint_images Celery task pipeline chain
- Connect EXIF metadata check -> ELA analysis -> HuggingFace detector
- Save AI confidence score and is_fake flag to Complaint model

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete the Celery AI validation pipeline chain.
Task: Implement complaints/tasks.py validate_complaint_images task.
1. Pipeline steps:
   - Step 1: Check EXIF tags (Pillow). If GPS/Camera tags found -> pass.
   - Step 2: Error Level Analysis (OpenCV). Calculate mean squared error.
   - Step 3: HuggingFace fake detector inference (umm-maybe/AI-image-detector).
2. If is_fake == True (confidence > 0.70):
   - Set complaint.status = 'Rejected', complaint.rejection_reason = 'AI detected artificial/downloaded image'.
3. If is_fake == False:
   - Set complaint.status = 'Validated', trigger route_complaint_task.
```

```
Generate complete Celery task and error-handling fallbacks.
```

DAY 11 Om Gadekar — Frontend Lead

KEY TASKS:

- Create WorkerLayout.jsx with mobile bottom navigation bar
- Build worker/TaskList.jsx placeholder with status tab filter
- Style priority-coded task cards with Tailwind CSS

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Scaffold the Worker Portal UI.
Task: Create WorkerLayout.jsx and worker/TaskList.jsx.
1. WorkerLayout.jsx:
   - Top Header: JANMITRA logo, 'Field Worker' badge, Notification bell, Logout.
   - Bottom Tab Bar (mobile-first): Tasks (active) | Completed | Profile.
2. worker/TaskList.jsx:
   - Filter tabs: Assigned (New) | In-Progress | Completed.
   - Task Card: Complaint ID, Category badge, Priority pill, Ward badge, Address, 'Open Task' button.
   - Empty state: 'No tasks currently assigned in your ward.'
Generate WorkerLayout.jsx and worker/TaskList.jsx completely.
```

DAY 11 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Create citizen/Submit.jsx complaint reporting form foundation
- Build 4 Category Selection cards with vibrant icons and colors
- Add Ward selection dropdown and Description textarea

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Citizen Issue Submission form (Part 1).
Task: Create citizen/Submit.jsx form layout.
1. Page Header: 'Report New Civic Issue' with back button to /citizen.
2. 4 Category Cards (grid 2x2):
   - [PWD] Roads & Infrastructure (Blue)
   - [SWM] Garbage & Waste (Green)
   - [PHE] Water & Drainage (Cyan)
   - [ELE] Street Lighting (Yellow)
   - Interactive selection with blue outline and checkmark.
3. Ward selector: Dropdown [Ward 1 | Ward 2 | Ward 3 | Ward 4] (pre-selected from user profile).
4. Description field: Textarea (min 15 chars) with character counter.
5. Landmark input: Optional text field for easy field location.
Generate complete citizen/Submit.jsx with responsive Tailwind CSS.
```

DAY 12 Anas Shaikh — Backend Lead

KEY TASKS:

- Create Worker Task API: GET /api/v1/worker/tasks/
- Filter tasks strictly by worker's registered department and ward
- Implement pagination and status filters (Assigned, In-Progress)

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Worker Task List API endpoint.
Task: Create WorkerTaskListView in complaints/views.py.
1. Endpoint: GET /api/v1/worker/tasks/
   - Permissions: IsAuthenticated, IsWorker
   - Automatically filters complaints where:
     department == request.user.department AND ward == request.user.ward
2. Query parameters:
   - ?status=assigned (or in_progress, completed)
   - ?priority=high (or critical, medium)
3. Return fields: id, complaint_id, category, description, photos, location_lat, location_lng, address, priority, status,
created_at.
4. Serializer: WorkerComplaintSerializer with optimized query.
Generate views.py and serializers.py additions.
```

DAY 12 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Write complaints/ai_module/duplicate_detector.py with PostGIS
- Implement ST_DWithin query for 100-metre radius proximity
- Link duplicate complaints to primary parent complaint

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the PostGIS duplicate complaint detector.
Task: Create duplicate_detector.py using GeoDjango PostGIS spatial queries.
1. duplicate_check(complaint):
   - Find existing complaints where:
     category == complaint.category AND
     status in ['Submitted', 'Validated', 'Routed', 'Assigned', 'In-Progress'] AND
     ST_DWithin(location, complaint.location, D(m=100))
2. If match found:
   - Set complaint.is_duplicate = True
   - Set complaint.parent_complaint = existing_complaint
   - Increment existing_complaint.duplicate_count by 1
   - Boost existing_complaint priority score by +15 points
Generate duplicate_detector.py with fallback for SQLite development mode.
```

DAY 12 Om Gadekar — Frontend Lead

KEY TASKS:

- Build worker/TaskDetail.jsx page layout
- Integrate Leaflet map showing complaint location pin and worker directions
- Add 'Start Work' action button transitioning status to In-Progress

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build worker/TaskDetail.jsx.
Task: Create worker/TaskDetail.jsx with GPS navigation.
1. Fetch: GET /api/v1/worker/tasks/:id/
2. Header: Complaint ID, PriorityBadge, WardBadge, Time elapsed since assignment.
3. Location Map: Leaflet map container showing pin, address, and 'Open in Google Maps' external link.
4. Issue Photos: Citizen uploaded photos gallery with pinch-to-zoom.
5. Action Buttons:
   - If status == 'Assigned': Blue 'Start Work' button (PATCH status to In-Progress).
   - If status == 'In-Progress': Amber 'Upload Proof & Complete' CTA.
Generate complete worker/TaskDetail.jsx.
```

DAY 12 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Add Photo Uploader to citizen/Submit.jsx
- Implement live camera capture and gallery upload (1 to 3 photos)
- Add real-time client-side image compression and preview carousel

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Add photo upload to Citizen Submission Form.
Task: Enhance citizen/Submit.jsx with photo uploader.
1. Photo Picker Component:
   - Take Photo button (opens native camera on mobile: accept='image/*' capture='environment')
   - Upload from Gallery button (accept='image/*')
   - Max 3 photos allowed with file size validation (< 5MB each).
2. Preview Gallery:
   - Thumbnail previews with delete (X) badge.
   - Primary photo badge on first photo.
3. Canvas Image Compression:
   - Client-side resize to max 1280px width to ensure fast upload on mobile 4G.
Generate updated citizen/Submit.jsx with photo capture handlers.
```

DAY 13 Anas Shaikh — Backend Lead

KEY TASKS:

- Create Worker Status Update API: PATCH /api/v1/worker/tasks/:id/status/
- Enforce status transition rules (Assigned -> In-Progress -> Work Done)
- Log every worker action to StatusHistory with worker timestamp

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Worker Status Update API.
Task: Create WorkerTaskStatusUpdateView in complaints/views.py.
1. Endpoint: PATCH /api/v1/worker/tasks/:id/status/
2. Validation rules:
   - Only assigned worker can update status.
   - 'In-Progress' allowed only from 'Assigned'.
   - 'Work Done' requires at least 1 proof-of-work photo in request.
3. Automatically creates StatusHistory entry:
   - changed_by = request.user, old_status, new_status, timestamp.
4. Triggers trigger_work_done notification to citizen when Work Done.
Generate views.py endpoint and serializer validation.
```

DAY 13 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Write complaints/ai_module/priority_engine.py
- Implement weighted formula: severity*0.30 + safety*0.25 + impact*0.20 + dupes*0.15 + dept*0.10
- Map numerical score (0-100) to Critical, High, Medium, Low

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the AI Priority Scoring Engine.
Task: Create complaints/ai_module/priority_engine.py.
1. Formula:
    Score = (severity * 0.30) + (public_safety * 0.25) + (traffic_impact * 0.20) + (duplicate_factor * 0.15) +
(sla_urgency * 0.10)
2. Priority levels:
   - Score >= 80 -> 'Critical' (Target SLA: 24h)
   - Score 60-79 -> 'High' (Target SLA: 48h)
   - Score 40-59 -> 'Medium' (Target SLA: 72h)
   - Score < 40  -> 'Low' (Target SLA: 7 days)
3. Priority calculation function: calculate_priority(complaint_data) -> {score: int, level: str}
Generate priority_engine.py with unit test cases.
```

DAY 13 Om Gadekar — Frontend Lead

KEY TASKS:

- Build Proof-of-Work photo upload modal in worker/TaskDetail.jsx
- Add camera capture for repaired site photos
- Add completion notes textarea and 'Mark Work Done' submit button

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Worker Proof-of-Work upload modal.
Task: Create ProofOfWorkModal component inside worker/TaskDetail.jsx.
1. Modal header: 'Upload Completion Proof' (Complaint #ID).
2. Camera uploader: capture live photo of resolved site (Mandatory 1-2 photos).
3. Repair Notes: Textarea for worker remarks (e.g., 'Pothole filled with cold mix bitumen').
4. Action: POST to /api/v1/worker/tasks/:id/proof/ with multipart/form-data.
5. On success: close modal, update status pill to 'Work Done', show success toast.
Generate modal component with Tailwind styling.
```

DAY 13 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Integrate Leaflet Map Location Picker into citizen/Submit.jsx
- Implement auto GPS geolocate button ('Use Current Location')
- Add draggable marker with latitude/longitude confirmation for Nagpur

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Add Leaflet GPS map picker to citizen/Submit.jsx.
Task: Add interactive LocationPicker to citizen/Submit.jsx.
1. Leaflet map centered at Nagpur (21.1458° N, 79.0882° E).
2. 'Detect My Location' button (calls navigator.geolocation.getCurrentPosition):
   - Places draggable marker at user's current GPS coordinates.
   - Reverse-geocodes or shows latitude & longitude badge.
3. Manual pin drag: User can drag pin to adjust exact location on street.
4. Accuracy indicator: Shows GPS accuracy radius in metres.
Generate LocationPicker component inside citizen/Submit.jsx.
```

DAY 14 Anas Shaikh — Backend Lead

KEY TASKS:

- Create Proof-of-Work upload API: POST /api/v1/worker/tasks/:id/proof/
- Save proof images to ComplaintImage model with type='proof'
- Update complaint status to 'Work Done'

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Proof-of-Work upload API.
Task: Create WorkerProofUploadView in complaints/views.py.
1. Endpoint: POST /api/v1/worker/tasks/:id/proof/
2. Accepts: multipart/form-data (proof_images, remarks).
3. Saves uploaded files into ComplaintImage model with image_type='proof'.
4. Sets complaint.status = 'Work Done', complaint.resolved_at = timezone.now().
5. Calls Celery task send_push_notification to citizen recipient.
Generate view and serializer with validation.
```

DAY 14 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Integrate YOLOv8 civic object detector in ai_module/yolo_validator.py
- Detect civic issue classes: pothole, garbage_dump, water_leakage, street_light_damage
- Set confidence threshold (0.50): if below threshold, flag for Human Validation queue

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Implement YOLOv8 civic issue validation.
Task: Create yolo_validator.py using Ultralytics YOLOv8.
1. Load model: YOLO('weights/janmitra_civic_yolov8n.pt') (with graceful fallback if model not found).
2. Run inference: results = model.predict(image_path, conf=0.50)
3. Check detected classes against expected complaint category.
4. If detected: return {valid: True, confidence: 0.88, detected_class: 'pothole'}
5. If low confidence: return {valid: False, flag_manual_review: True, reason: 'Low AI confidence'}
Generate yolo_validator.py with fallback simulation mode.
```

DAY 14 Om Gadekar — Frontend Lead

KEY TASKS:

- Build worker/CompletedTasks.jsx archive page
- Display completed repair history with before/after photo comparison
- Add search by complaint ID and date range filter

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build worker/CompletedTasks.jsx.
Task: Create worker/CompletedTasks.jsx archive screen.
1. Fetch: GET /api/v1/worker/tasks/?status=completed
2. Layout: Grid/list of resolved tasks.
```

```
3. Card shows: Complaint ID, Ward, Resolution date, Before & After photos thumbnail pill.
4. 'View Details' modal: Shows full resolution summary and citizen verification status.
Generate worker/CompletedTasks.jsx with Tailwind CSS.
```

DAY 14 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Connect citizen/Submit.jsx to POST /api/v1/complaints/ API
- Handle multipart form upload with loading progress spinner
- Show Success Modal with Complaint ID and 'Track Complaint' button

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete Citizen Complaint Submission flow.
Task: Wire citizen/Submit.jsx to backend API.
1. Submit action: POST /api/v1/complaints/ with FormData (category, ward, description, photos, lat, lng).
2. Loading state: Animated pulse loader with 'Uploading photos & running AI verification...'.
3. Success Modal:
   - Green checkmark animation.
   - Header: 'Complaint Registered Successfully!'.
   - Ticket badge: Complaint ID (e.g., NMC-2026-018).
   - Expected SLA: 'Target resolution within 48 hours'.
   - Button: 'Track Issue Status' -> routes to /citizen/complaint/:id.
Generate complete submission submission handlers and modal.
```

DAY 15 Anas Shaikh — Backend Lead

KEY TASKS:

- Create Citizen Verification API: POST /api/v1/complaints/:id/verify/
- Handle Accept action: sets status='Closed' and records 1-5 star rating
- Handle Reject action: sets status='Reopened' and logs citizen rework reason

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Citizen Verification API.
Task: Create CitizenComplaintVerifyView in complaints/views.py.
1. Endpoint: POST /api/v1/complaints/:id/verify/
2. Request payload:
   - action: 'accept' or 'reject'
   - rating: integer 1-5 (if accept)
   - reason: string (if reject)
3. If accept: complaint.status = 'Closed', complaint.rating = rating, complaint.closed_at = timezone.now().
4. If reject: complaint.status = 'Reopened', complaint.reopen_count += 1, routes back to worker queue.
Generate view and serializer with validation.
```

DAY 15 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Write Celery periodic task auto_close_unverified_complaints in ai_module/tasks.py
- Query complaints in 'Work Done' status for more than 72 hours without citizen response
- Auto-close and award default 5-star resolution rating

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build auto-close periodic task.
Task: Create auto_close_unverified_complaints in notifications/tasks.py.
1. Celery periodic task runs every 6 hours.
2. Query: Complaint.objects.filter(status='Work Done', resolved_at__lte=now - 72h).
3. For each complaint:
   - complaint.status = 'Closed'
   - complaint.auto_closed = True
   - complaint.rating = 5 (default civic compliance)
   - Log StatusHistory: 'Auto-closed after 72 hours of citizen inactivity'.
4. Triggers push notification to citizen informing them of auto-close.
Generate Celery periodic task and Celery Beat schedule entry.
```

DAY 15 Om Gadekar — Frontend Lead

KEY TASKS:

- Build worker/Profile.jsx profile and shift management page
- Display worker NMC employee ID, registered department, ward, and active task count
- Add toggle for 'On Duty' / 'On Leave' status with API sync

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build worker/Profile.jsx.
Task: Create worker/Profile.jsx screen.
1. Profile Header: Worker Avatar, Full Name, Designation (e.g. Field Inspector), Employee ID.
2. Assignment details: Department (e.g. PWD), Municipal Ward (e.g. Ward 2).
3. Status toggle: 'Active On-Duty' vs 'On Leave' (syncs to PATCH /api/v1/worker/profile/).
4. Summary metrics: Total Tasks Completed, Average Turnaround (Days), Rating received.
5. Logout button: Clears AuthContext and redirects to /login.
Generate worker/Profile.jsx completely.
```

DAY 15 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Connect Citizen Verification Card in citizen/ComplaintDetail.jsx
- Render 5-Star Rating Modal when user clicks 'Accept & Close'
- Render Rework Reason Modal when user clicks 'Reject & Reopen'

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete Citizen Verification UI in ComplaintDetail.jsx.
Task: Wire Citizen Verification flow in citizen/ComplaintDetail.jsx.
1. When complaint status is 'Work Done':
   - Display Citizen Verification Box with proof-of-work photo thumbnails.
   - Dual CTAs: 'Verify & Accept Resolution' (Green) | 'Reject / Issue Not Fixed' (Red).
2. Accept Modal: 5-Star rating selector with star hover micro-animation and feedback comment.
3. Reject Modal: Reason selector dropdown (e.g. 'Pothole still visible', 'Garbage not cleared') and note.
4. API call: POST /api/v1/complaints/:id/verify/.
Generate modal handlers and rating component.
```

DAY 16 Anas Shaikh — Backend Lead

KEY TASKS:

- Create StatusHistory audit model and API endpoint: GET /api/v1/complaints/:id/timeline/
- Ensure every lifecycle event is immutable and logged with timestamp and user role
- Return chronological timeline for citizen stepper

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build StatusHistory Timeline API.
Task: Create StatusHistory model and timeline view.
1. StatusHistory model:
   - complaint (FK), old_status, new_status, changed_by (FK User), remarks, created_at.
2. Endpoint: GET /api/v1/complaints/:id/timeline/
3. Returns array of timeline events sorted chronologically:
   [{step: 'Submitted', timestamp: '...', actor: 'Citizen'}, {step: 'Validated', ...}]
Generate models.py, serializers.py, and views.py.
```

DAY 16 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Build Dashboard KPI Aggregation Queries in dashboard/views.py
- Calculate Total Complaints, Pending Triage, Resolved, and Average TAT per department
- Add department filter query parameter (?dept_id=1)

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Dashboard KPI aggregation queries.
Task: Create DashboardKPIView in dashboard/views.py.
1. Endpoint: GET /api/v1/dashboard/kpi/
2. Query params: ?dept_id=1 (or without dept_id for cross-department summary).
```

```
3. Aggregate queries:
   - total = count()
   - pending = count(status in ['Submitted', 'Validated', 'Routed'])
   - in_progress = count(status in ['Assigned', 'In-Progress'])
   - resolved = count(status in ['Work Done', 'Closed'])
   - avg_days = Avg(resolved_at - created_at)
Generate views.py with optimized database aggregation.
```

DAY 16 Om Gadekar — Frontend Lead

KEY TASKS:

- Build Pending Human Validation Queue UI in officer/ValidationQueue.jsx
- Display AI-flagged edge cases with low confidence scores
- Provide side-by-side photo inspection and manual Approve / Reject buttons

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build officer/ValidationQueue.jsx.
Task: Create officer/ValidationQueue.jsx for manual AI review.
1. Fetch: GET /api/v1/dashboard/validation-queue/
2. Layout: Card queue of complaints flagged by AI with confidence score < 0.50.
3. Card shows: Uploaded photo, AI confidence bar, category, ward, citizen description.
4. Actions:
   - 'Approve' (Green): Validates complaint and routes to field team.
   - 'Reject' (Red): Marks as invalid with reason selection.
Generate officer/ValidationQueue.jsx completely.
```

DAY 16 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Add Fullscreen Lightbox Photo Modal to citizen/ComplaintDetail.jsx
- Support multi-image carousel navigation (Next/Prev) with zoom
- Differentiate before-repair photos from worker proof-of-work photos

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Add Image Lightbox to ComplaintDetail.jsx.
Task: Create PhotoLightboxModal component in citizen/ComplaintDetail.jsx.
1. When any complaint image is clicked, open full-screen lightbox modal.
2. Displays image badge: 'Original Report' vs 'Proof of Work'.
3. Next/Previous navigation buttons for multi-image complaints.
4. Keyboard support: ESC to close, Left/Right arrow keys to navigate.
Generate PhotoLightboxModal component.
```

DAY 17 Anas Shaikh — Backend Lead

KEY TASKS:

- Build Department Assignment API: POST /api/v1/complaints/:id/assign/
- Enforce strict ward and department rule: worker must match complaint dept and ward
- Notify assigned worker via push trigger

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Worker Assignment API.
Task: Create ComplaintAssignWorkerView in complaints/views.py.
1. Endpoint: POST /api/v1/complaints/:id/assign/
2. Payload: { worker_id: UUID }
3. Validation:
   - Worker.department == complaint.department
   - Worker.ward == complaint.ward
   - (Admin can override ward rule if override_reason is provided)
4. Updates complaint.assigned_worker = worker, complaint.status = 'Assigned'.
5. Triggers trigger_task_assigned push notification.
Generate view and serializer.
```

DAY 17 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Build Ward-by-Department Statistics API in dashboard/views.py
- Generate 4x4 density matrix for Wards 1 to 4 across all 4 departments
- Format data for the municipal Heatmap table

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Ward-by-Department Heatmap API.
Task: Create WardStatsView in dashboard/views.py.
1. Endpoint: GET /api/v1/dashboard/ward-stats/
2. Returns 4x4 matrix:
   Rows: Ward 1, Ward 2, Ward 3, Ward 4.
   Columns: PWD, SWM, PHE, Electrical counts.
3. Supports ?dept_id=X for single department ward breakdown.
Generate view with aggregation queries.
```

DAY 17 Om Gadekar — Frontend Lead

KEY TASKS:

- Enhance officer/Dashboard.jsx with Recharts category bar chart
- Render 4 civic categories volume (Roads, Garbage, Water, Lighting)
- Add Ward issue density progress meters

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Enhance officer/Dashboard.jsx with Recharts.
Task: Update officer/Dashboard.jsx with charts.
1. Recharts BarChart: X=Categories (Roads, Garbage, Water, Lighting), Y=Total complaints.
2. Color-coded bars matching department themes (Blue, Green, Cyan, Yellow).
3. Ward distribution progress meters for Wards 1 to 4.
4. Quick actions toolbar: 'Dispatch Workers', 'Review AI Validations'.
Generate updated officer/Dashboard.jsx.
```

DAY 17 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Implement FCM Push Notification Triggers in notifications/triggers.py
- Wire triggers for: complaint_submitted, complaint_validated, task_assigned, work_done, complaint_closed
- Format deep links and sound alerts for mobile and desktop

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise FCM push triggers.
Task: Complete notifications/triggers.py.
1. trigger_complaint_submitted(complaint)
2. trigger_complaint_validated(complaint)
3. trigger_task_assigned(complaint, worker)
4. trigger_work_done(complaint)
5. trigger_complaint_closed(complaint)
Each trigger queues Celery task send_push_notification with title, body, and click_action.
Generate triggers.py with logging and error guards.
```

DAY 18 Anas Shaikh — Backend Lead

KEY TASKS:

- Implement SLA breach detection service and overdue task flagger
- Calculate elapsed hours against target turnaround time
- Expose /api/v1/dashboard/overdue/ endpoint for officers

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build SLA Breach Tracking system.
Task: Create SLA tracking logic in complaints/services.py.
1. SLA Targets: Critical=24h, High=48h, Medium=72h, Low=7 days.
2. Function get_overdue_complaints(): Returns complaints exceeding target SLA.
3. Add is_overdue boolean property on Complaint model.
```

```
4. API: GET /api/v1/dashboard/overdue/?dept_id=X.
Generate service function and API view.
```

DAY 18 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Build category-wise complaint volume distribution endpoints
- Calculate department turnaround time (TAT) moving average
- Optimize spatial index queries for 4 NMC ward boundaries

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Department Analytics backend.
Task: Enhance dashboard/views.py with analytics queries.
1. Moving average resolution time query for last 30 days.
2. Category-wise resolution success rate percentage.
3. PostGIS spatial cluster index optimization for Ward 1-4.
Generate view methods with caching.
```

DAY 18 Om Gadekar — Frontend Lead

KEY TASKS:

- Build High Priority Complaints Queue component for officer portal
- Display urgent civic issues sorted by priority score and SLA risk
- Add one-click 'Quick Assign' worker modal

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build HighPriorityQueue component.
Task: Create HighPriorityQueue.jsx component for Officer Dashboard.
1. Table/card list of complaints with priority Critical and High.
2. Shows: Complaint ID, Ward, SLA Time Remaining (countdown pill), Category.
3. 'Assign Worker' button: Opens quick modal showing available on-duty workers for that ward.
Generate HighPriorityQueue component.
```

DAY 18 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Add deep-linking and browser focus handler for push notifications
- Ensure clicking background notifications navigates to /citizen/complaint/:id
- Test across desktop Chrome, Edge, and mobile browser PWA

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Add push notification deep-linking.
Task: Update firebase-messaging-sw.js and FirebaseProvider.jsx.
1. Service Worker notificationclick handler:
   - Focuses existing JANMITRA tab if already open, or opens new window.
   - Deep-links directly to target complaint URL from payload.fcmOptions.link.
2. In-app Toast click handler: Navigates cleanly using React Router useNavigate.
Generate updated service worker and provider navigation code.
```

DAY 19 Anas Shaikh — Backend Lead

KEY TASKS:

- Database optimization: Add composite indexes for status, department, and ward
- Add select_related and prefetch_related across all complaint queries
- Benchmark API response times (< 100ms)

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Optimize database queries and indexes.
Task: Update models.py indexes and views.py querysets.
1. Add composite indexes on Complaint model:
   - db_index on [department, ward, status]
   - db_index on [citizen, -created_at]
2. Refactor ComplaintListView:
   - Use select_related('citizen', 'department', 'ward', 'assigned_worker')
```

```
   - Use prefetch_related('images', 'history')
3. Verify query count drops from N+1 to 2 queries per request.
Generate migration and query optimizations.
```

DAY 19 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Tune AI confidence thresholds and error fallbacks
- Benchmark inference speed (< 400ms per image)
- Write test suite for AI pipeline error handling with corrupted images

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Optimize AI Pipeline and test error handling.
Task: Create ai_module/tests.py and optimize inference.
1. Test corrupted/truncated image files: ensure pipeline logs warning without crashing Celery.
2. Test AI edge case images: ensure low-confidence items route to Validation Queue.
3. Add memory cleanup: cv2.destroyAllWindows() and gc.collect() after each batch.
Generate tests.py for AI module.
```

DAY 19 Om Gadekar — Frontend Lead

KEY TASKS:

- Implement offline task caching in localStorage for worker portal
- Enable field workers to view assigned task details without active internet
- Add offline indicator pill and auto-sync when connection is restored

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Add offline caching to Worker Portal.
Task: Implement offline storage in worker/TaskList.jsx.
1. Cache assigned tasks in localStorage on successful fetch.
2. If navigator.onLine is false: render cached tasks with amber 'Offline Mode' pill.
3. Queue status updates in IndexedDB/localStorage when offline.
4. Auto-sync queued updates when online event fires.
Generate offline cache utility and worker component update.
```

DAY 19 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Create citizen/Profile.jsx with masked mobile, inline edit, and account stats
- Add Change Password form with show/hide toggle
- Add Logout button clearing AuthContext and localStorage

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build citizen/Profile.jsx.
Task: Create citizen/Profile.jsx screen.
1. Header: User Avatar, Full Name with 'Edit Name' inline button, Masked Mobile (***3210), Ward pill.
2. My Stats cards: Total filed, Resolved, In-Progress, Average rating given (4.8 *).
3. Change Password form: Current password, New password (min 8 chars), Confirm password with show/hide toggle.
4. Logout button: Red button at bottom clearing AuthContext and redirecting to /login.
Generate citizen/Profile.jsx completely.
```

DAY 20 Anas Shaikh — Backend Lead

KEY TASKS:

- Phase 2 Backend Integration Audit: Test all 20 API endpoints end-to-end
- Verify RBAC permissions for all 4 roles across every route
- Generate OpenAPI / Swagger schema documentation

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete Phase 2 Backend Audit.
Task: Run complete API integration verification.
1. Verify endpoints:
   - Auth: register, login, profile, fcm-token
   - Complaints: submit, list, detail, assign, status, proof, verify, timeline
```

```
   - Worker: tasks, task-detail, status-update
   - Dashboard: kpi, ward-stats, overdue
2. Verify RBAC permission blocks unauthorized access (e.g. citizen cannot access worker endpoints).
3. Write test script test_all_endpoints.py.
Generate integration test script.
```

DAY 20 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Finalise AI validation metrics report (Accuracy, False Positive rate)
- Document PostGIS duplicate detection distance benchmarks
- Prepare AI module architecture diagrams for project documentation

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete AI Module Documentation.
Task: Create AI_MODULE_REPORT.md.
1. Document Layer 1 (EXIF + ELA + HuggingFace) accuracy benchmarks.
2. Document Layer 2 (YOLOv8) civic issue detection performance on Nagpur sample images.
3. Document PostGIS ST_DWithin 100m duplicate detection algorithm and execution time (< 15ms).
Generate complete markdown documentation.
```

DAY 20 Om Gadekar — Frontend Lead

KEY TASKS:

- Phase 2 Frontend UI Audit: Test mobile responsiveness across all devices
- Verify bottom navigation active highlights and smooth page transitions
- Fix any UI glitches in Worker and Officer screens

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete Phase 2 Frontend Polish.
Task: Audit and polish all portal layouts.
1. Test responsive layouts on mobile (375px, 414px), tablet (768px), and desktop (1280px).
2. Verify Citizen bottom nav, Worker bottom nav, and Officer sidebar drawers.
3. Check loading skeleton states across all data-fetching pages.
Generate UI polish fixes and Tailwind updates.
```

DAY 20 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Build citizen/Help.jsx with accordion FAQs and Nagpur civic helplines
- Add search bar filtering questions and answers
- Update CitizenLayout to 4 bottom tabs: Home | My Complaints | Help | Profile

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build citizen/Help.jsx and 4-tab bottom nav.
Task: Create citizen/Help.jsx and update CitizenLayout.jsx.
1. citizen/Help.jsx:
   - Header: 'How to Use JANMITRA' -- NMC Helpdesk.
   - Search filter bar.
   - Accordion FAQ cards: How to report, Photo rejection reasons, Tracking, SLA times, Reopening issues.
   - NMC Civic Helplines: Toll-Free 1800-233-3766, SWM, Water, Electrical phone numbers.
2. Update CitizenLayout.jsx:
   - 4 Tabs: Home | My Complaints | Help | Profile (active blue #1A56DB).
Generate citizen/Help.jsx and updated CitizenLayout.jsx.
```

Phase 3 -- Department Dashboard & Analytics — Days 21 -- 28

DAY 21 Anas Shaikh — Backend Lead

KEY TASKS:

- Create Worker Management API: GET and POST /api/v1/admin/workers/
- Implement employee ID generation (e.g. NMC-PWD-101)
- Add department officer permission restrictions

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Worker Management API.
Task: Create AdminWorkersListCreateView in accounts/views.py.
1. GET /api/v1/admin/workers/ with search, department, ward, and status filters.
2. POST /api/v1/admin/workers/ creates worker account with employee_id, mobile, ward, department, designation.
3. If user is Dept. Officer: lock department to request.user.department only.
Generate views, serializers, and permission checks.
```

DAY 21 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Build Ward Heatmap aggregation query in dashboard/views.py
- Group complaints count by Ward 1-4 and 4 departments
- Add severity level filtering (Critical, High, Medium, Low)

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Ward Density Heatmap API.
Task: Enhance WardStatsView in dashboard/views.py.
1. Aggregate complaint counts grouped by (ward, department).
2. Compute ward total, department total, and city total.
3. Return normalized density score (0.0 to 1.0) for color heatmaps.
Generate query and serializer.
```

DAY 21 Om Gadekar — Frontend Lead

KEY TASKS:

- Build admin/WorkerManagement.jsx table view
- Display Employee ID, Name, Dept, Ward, Active Tasks, Status
- Add search bar and filter dropdowns (Dept, Ward, Status)

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build admin/WorkerManagement.jsx.
Task: Create admin/WorkerManagement.jsx screen.
1. Page Title: 'Worker Management' with blue 'Add New Worker' button.
2. Filter bar: Search by name/ID, Department [All|PWD|SWM|PHE|Electrical], Ward [All|1|2|3|4], Status [All|Active|Inactive].
3. Table columns: Employee ID | Name | Dept | Ward | Designation | Active Tasks | Status | Actions (Edit, Toggle Status).
4. Pagination: 20 per page.
Generate admin/WorkerManagement.jsx with Tailwind CSS.
```

DAY 21 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Build WorkerRegistrationForm.jsx modal component
- Form fields: Employee ID, Full Name, Mobile, Email, Dept, Ward, Designation, Password
- Add auto-lock department field for Dept. Officers

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build WorkerRegistrationForm.jsx modal.
Task: Create WorkerRegistrationForm.jsx modal.
1. Mode: Create new worker (POST) or Edit existing worker (PATCH).
2. Input fields with validation:
   - Employee ID (required, NMC format)
   - Full Name (required)
   - Mobile Number (required, 10 digits)
   - Department dropdown (locked if logged-in user is officer)
   - Ward dropdown (Ward 1 to 4)
```

```
   - Designation text input
   - Initial Password & Confirm Password
3. On submit: call API, show success toast, refresh worker list.
Generate WorkerRegistrationForm.jsx.
```

DAY 22 — Anas Shaikh — Backend Lead

KEY TASKS:

- Build Worker Detail and Edit API: PATCH /api/v1/admin/workers/:id/
- Allow updating designation, ward, department (admin only), and status
- Protect mobile and employee_id from modification

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Worker Update API.
Task: Create AdminWorkerDetailUpdateView in accounts/views.py.
1. Endpoint: PATCH /api/v1/admin/workers/:id/
2. Editable fields: designation, ward, status (Active/Inactive/On Leave).
3. Immutable fields: employee_id, mobile cannot be changed after creation.
4. Department field: editable by Super Admin only; read-only for Dept. Officers.
Generate view and validation serializer.
```

DAY 22 — Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Build Pending Human Validation Queue API endpoint
- Filter complaints where AI confidence score < 0.50
- Return AI detection tags, confidence score, and uploaded photo URL

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Validation Queue API.
Task: Create ValidationQueueView in dashboard/views.py.
1. Endpoint: GET /api/v1/dashboard/validation-queue/
2. Query: Complaint.objects.filter(status='Pending Validation', ai_confidence__lt=0.50)
3. Supports ?dept_id=X filter for department officers.
4. Return fields: id, complaint_id, category, description, photo_url, ai_detected_tags, ai_confidence, created_at.
Generate view and serializer.
```

DAY 22 — Om Gadekar — Frontend Lead

KEY TASKS:

- Connect Edit Worker Modal and Status Toggle in admin/WorkerManagement.jsx
- Add confirmation dialog for worker deactivation
- Show real-time active tasks count badge

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Connect Worker Edit and Status Toggle.
Task: Update admin/WorkerManagement.jsx with edit and toggle actions.
1. Edit button: Opens WorkerRegistrationForm pre-filled with worker data.
2. Status Toggle: Click toggle switches between Active and Inactive (calls PATCH).
3. Confirmation dialog before deactivating workers with active assigned tasks.
4. Active tasks badge: Green if 0, Amber if 1-3, Red if > 4 tasks.
Generate updated WorkerManagement.jsx.
```

DAY 22 — Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Enhance NotificationBell.jsx with 60-second live polling interval
- Add click-outside dismissal and unread counter badge pulse
- Implement mark-all-as-read action

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Enhance NotificationBell component.
Task: Update src/components/NotificationBell.jsx.
1. 60-second periodic poll to GET /api/v1/notifications/ to fetch new alerts.
```

```
2. Badge counter with pulse animation when unread_count > 0.
3. Dropdown menu:
   - Header with 'Mark all read' button (PATCH /api/v1/notifications/mark-read/).
   - Notification item list with time ago (e.g. '5m ago').
   - Click item marks read and deep-links to complaint.
   - Click-outside dismisses dropdown.
Generate updated NotificationBell.jsx.
```

DAY 23 Anas Shaikh — Backend Lead

KEY TASKS:

- Create Complaints Export API: GET /api/v1/dashboard/export/
- Support CSV and Excel export formats with date range filter
- Include all complaint lifecycle timestamps and assigned worker details

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Complaints Export API.
Task: Create ComplaintsExportView in dashboard/views.py.
1. Endpoint: GET /api/v1/dashboard/export/?format=csv&start_date=...&end_date=...
2. Permissions: IsAuthenticated, IsDeptOfficer or IsAdmin.
3. Generates streaming CSV with columns:
   Complaint ID, Category, Ward, Status, Priority, Citizen Mobile, Assigned Worker,
   Submitted At, Resolved At, Closed At, Citizen Rating.
Generate export view with streaming response.
```

DAY 23 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Build Manual AI Validation Override API: POST /api/v1/dashboard/validate/:id/
- Accept action: 'approve' (validates and routes) or 'reject' (rejects complaint)
- Log officer manual override note to StatusHistory

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Validation Override API.
Task: Create ManualValidationOverrideView in dashboard/views.py.
1. Endpoint: POST /api/v1/dashboard/validate/:id/
2. Payload: { action: 'approve' | 'reject', remarks: str }
3. If approve: complaint.status = 'Validated', trigger routing task.
4. If reject: complaint.status = 'Rejected', complaint.rejection_reason = remarks.
5. Create StatusHistory entry noting manual human validation by officer.
Generate view and serializer.
```

DAY 23 Om Gadekar — Frontend Lead

KEY TASKS:

- Build Super Admin Cross-Department Dashboard (admin/CrossDeptDashboard.jsx)
- Top bar Department Switcher: [All Departments | PWD | Solid Waste | Water Works | Electrical]
- Render 4 KPI summary cards with department color accents

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build admin/CrossDeptDashboard.jsx (Part 1).
Task: Create admin/CrossDeptDashboard.jsx layout and KPI cards.
1. Page Title: 'JANMITRA Admin Dashboard' -- 'All Departments Overview'.
2. Top Department Switcher tabs:
   - All Departments (active)
   - PWD (Blue), Solid Waste (Green), Water Works (Cyan), Electrical (Yellow).
3. 4 Cross-Dept KPI Cards in a row:
   - Each shows: Dept Name, Total, Pending, Resolved, Avg. Days.
   - Accents: PWD Blue, SWM Green, PHE Cyan, Electrical Yellow.
4. Fetch: GET /api/v1/dashboard/kpi/?dept_id=X for each department.
Generate CrossDeptDashboard.jsx structure.
```

DAY 23 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Add Recharts Category Bar Chart to admin/CrossDeptDashboard.jsx
- X axis: Roads, Garbage, Water, Street Lighting
- Bars: Total complaints with department thematic colors and custom tooltip

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Add Recharts Bar Chart to Admin Dashboard.
Task: Add Category Bar Chart to admin/CrossDeptDashboard.jsx.
1. ResponsiveContainer with BarChart (height 280px).
2. XAxis: 4 categories (Roads, Garbage, Water, Street Lighting).
3. Bar: dataKey='total' with rounded top corners (radius=[6, 6, 0, 0]).
4. Cell fills: #1A56DB (PWD), #16A34A (SWM), #0891B2 (PHE), #D97706 (Electrical).
5. Custom Tooltip showing category name, total, pending, and resolved counts.
Generate chart integration in CrossDeptDashboard.jsx.
```

DAY 24 Anas Shaikh — Backend Lead

KEY TASKS:

- Create Master Category & Ward boundaries configuration API
- Seed static categories (4) and wards (4) via Django data migration
- Expose /api/v1/meta/wards/ and /api/v1/meta/categories/ endpoints

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Metadata Master API and migration.
Task: Create metadata endpoints and data seed migration.
1. Seed 4 Categories: Roads & Infrastructure, Garbage & Waste, Water & Drainage, Street Lighting.
2. Seed 4 Wards: Ward 1 (Central), Ward 2 (North), Ward 3 (East), Ward 4 (West).
3. Seed 4 Departments: PWD, Solid Waste Management, Public Health Engineering, Electrical.
4. Read-only API endpoints for frontend dropdowns:
   - GET /api/v1/meta/categories/
   - GET /api/v1/meta/wards/
Generate migration and API views.
```

DAY 24 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Build Time-Series Trend Analysis API: GET /api/v1/dashboard/trends/
- Calculate daily submitted vs resolved complaint counts over 30 days
- Format data for Recharts LineChart

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Time-Series Trend API.
Task: Create DashboardTrendView in dashboard/views.py.
1. Endpoint: GET /api/v1/dashboard/trends/?days=30&dept_id=...
2. Query: Truncate date(created_at) and date(resolved_at) for past 30 days.
3. Return: [{ date: '2026-10-01', submitted: 14, resolved: 11 }, ...]
4. Include 7-day moving average.
Generate view and serializer.
```

DAY 24 Om Gadekar — Frontend Lead

KEY TASKS:

- Build Ward Heatmap table in admin/CrossDeptDashboard.jsx
- 4x4 Matrix: Rows (Ward 1-4) x Columns (PWD, SWM, PHE, Electrical)
- Cell color coding: Green (< 25 Low), Amber (25-42 Moderate), Red (> 42 Critical)

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Ward Heatmap Table.
Task: Add Ward Heatmap Table to admin/CrossDeptDashboard.jsx.
1. Rows: Ward 1, Ward 2, Ward 3, Ward 4.
2. Columns: PWD, SWM, PHE, Electrical, plus Ward Total.
3. Dynamic background and text colors based on complaint volume:
```

```
   - < 25: Green (Low activity)
   - 25-42: Amber (Moderate activity)
   - > 42: Red (Critical density)
4. Footer row: Department Total and City Total.
5. Legend: Color dots with volume thresholds.
Generate heatmap table in CrossDeptDashboard.jsx.
```

DAY 24 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Add Administrative Quick Actions to admin/CrossDeptDashboard.jsx
- Quick Links: Register New Worker, View All Complaints, Pending Validation
- Wire Department Switcher to render Officer Dashboard when department selected

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete Admin Dashboard Quick Actions.
Task: Add Quick Links and Switcher toggle to CrossDeptDashboard.jsx.
1. Quick Links cards (3 grid):
   - 'Register New Worker' -> /admin/workers?action=new
   - 'View All Complaints' -> /admin/complaints
   - 'Pending Validation (All Depts)' -> /officer/validation
2. Department Switcher logic:
   - When 'All Departments' selected: renders Cross-Dept overview (KPIs + Chart + Heatmap).
   - When specific department selected (PWD/SWM/PHE/Elec): renders officer/Dashboard.jsx for that department.
Generate updated CrossDeptDashboard.jsx.
```

DAY 25 Anas Shaikh — Backend Lead

KEY TASKS:

- Build Admin System Settings API: GET and PATCH /api/v1/admin/settings/
- Configure SLA threshold hours per priority level
- Add emergency civic broadcast alert setting

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build System Settings API.
Task: Create SystemSettingsView in dashboard/views.py.
1. Endpoint: GET and PATCH /api/v1/admin/settings/
2. Permissions: IsAdmin only.
3. Settings fields:
   - sla_critical_hours (default 24)
   - sla_high_hours (default 48)
   - sla_medium_hours (default 72)
   - auto_close_hours (default 72)
   - broadcast_message (string for citizen app banner)
Generate model, serializer, and view.
```

DAY 25 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Build Civic Health Index algorithm across Nagpur Wards
- Calculate composite score (0-100) based on resolution rate, SLA compliance, and backlog
- Expose /api/v1/dashboard/civic-health/ endpoint

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Civic Health Index algorithm.
Task: Create CivicHealthIndexView in dashboard/views.py.
1. Health Score Formula per Ward:
   Score = (ResolutionRate * 0.40) + (SLACompliance * 0.35) + ((100 - BacklogRatio) * 0.25)
2. Return rating for each ward: Excellent (> 85), Good (70-84), Needs Attention (< 70).
3. API: GET /api/v1/dashboard/civic-health/.
Generate algorithm service and endpoint.
```

DAY 25 Om Gadekar — Frontend Lead

KEY TASKS:

- Build Worker Roster & Workload Distribution View for department officers
- Display on-duty workers, current active tasks, and completion rate
- Add filter by municipal ward

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Worker Roster View.
Task: Create WorkerRoster.jsx component inside officer portal.
1. List of workers in officer's department.
2. Columns: Worker Name, Assigned Ward, Status (On Duty / On Leave), Active Tasks, Rating.
3. Workload meter bar: Green (1-2 tasks), Amber (3-4 tasks), Red (5+ tasks overloaded).
4. Direct 'Reassign Task' button.
Generate WorkerRoster.jsx component.
```

DAY 25 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Add In-App Broadcast Alert Banner to Citizen Home page
- Fetch emergency civic announcements from backend settings
- Add dismissible state cached in sessionStorage

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Add Citizen Broadcast Banner.
Task: Add BroadcastBanner component to citizen/Home.jsx.
1. Fetch broadcast_message from /api/v1/admin/settings/ or announcements API.
2. If active announcement exists: render amber banner at top of Citizen Home:
   - Speaker icon, title: 'NMC Civic Advisory', text, dismiss button (X).
3. Dismissing caches banner ID in sessionStorage to prevent repeated popups.
Generate BroadcastBanner component in citizen/Home.jsx.
```

DAY 26 Anas Shaikh — Backend Lead

KEY TASKS:

- Build City-Wide Complaints Ledger API: GET /api/v1/admin/complaints/
- Support multi-parameter filtering: department, ward, status, date range, search
- Include pagination (25 per page) and CSV export action

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Admin Complaints Ledger API.
Task: Create AdminComplaintsLedgerView in complaints/views.py.
1. Endpoint: GET /api/v1/admin/complaints/
2. Filters: department, ward, status, priority, date_from, date_to, search.
3. Full serialization including citizen name, mobile, assigned worker, and resolution time.
4. Response includes counts summary: total, open, in_progress, resolved.
Generate view and serializer.
```

DAY 26 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Build Predictive SLA Bottleneck detection query
- Identify wards or departments with rising backlogs before SLA breaches occur
- Generate alert warnings for department head

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build SLA Bottleneck Predictor.
Task: Create BottleneckDetectorService in dashboard/services.py.
1. Compare complaint arrival rate vs completion rate for past 7 days.
2. If arrival rate > completion rate by 1.5x: trigger Bottleneck Warning flag.
3. Include estimated hours until backlog breaches target SLA.
Generate service and API response integration.
```

DAY 26 Om Gadekar — Frontend Lead

KEY TASKS:

- Build admin/ComplaintsLedger.jsx city-wide complaints table
- Provide search by complaint ID, citizen mobile, and keyword
- Add multi-select status and department filter badges

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build admin/ComplaintsLedger.jsx.
Task: Create admin/ComplaintsLedger.jsx screen.
1. Title: 'City-Wide Complaints Ledger' -- Central NMC Database.
2. Filter bar: Search input, Department filter, Ward filter, Status tabs.
3. Table columns: Ticket ID, Date, Category, Ward, Citizen, Priority, Status, Assigned To, Action.
4. 'View Timeline' button: Opens stepper modal for that complaint.
Generate admin/ComplaintsLedger.jsx completely.
```

DAY 26 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Add Citizen Rating Summary and Review Feedback list in citizen/Profile.jsx
- Display average rating awarded to municipal teams
- Add feedback submission history

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Enhance citizen/Profile.jsx with ratings.
Task: Update citizen/Profile.jsx.
1. Render star rating visual breakdown (5*, 4*, 3*, 2*, 1* distribution).
2. Display citizen's past review remarks for resolved complaints.
3. Add 'NMC Citizen Satisfaction Score' badge.
Generate updated citizen/Profile.jsx.
```

DAY 27 Anas Shaikh — Backend Lead

KEY TASKS:

- Phase 3 Security & Permissions Audit across all endpoints
- Verify token expiration handling and refresh token rotation
- Ensure CORS policy is restricted to allowed frontend origins

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Complete Security & RBAC Audit.
Task: Audit all API endpoints and settings.
1. Verify DRF SimpleJWT token refresh rotation: ROTATE_REFRESH_TOKENS = True.
2. Verify CORS_ALLOWED_ORIGINS reads strictly from environment variables.
3. Test permission denial returns 403 Forbidden with clean JSON error message.
Generate security test script test_security_rbac.py.
```

DAY 27 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Complete Colab script for fine-tuning YOLOv8 on civic dataset
- Document model training hyperparameters and mAP scores
- Save trained model weights weights/janmitra_civic_yolov8n.pt

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise YOLO fine-tuning pipeline.
Task: Create ai_module/train_yolo_colab.py script.
1. Google Colab script to train YOLOv8n on civic dataset (pothole, garbage, water leak, streetlight).
2. Hyperparameters: epochs=50, imgsz=640, batch=16.
3. Output mAP@0.5 and mAP@0.5:0.95 metrics.
4. Export model to ONNX and PyTorch formats.
Generate complete training script with dataset loader.
```

DAY 27 Om Gadekar — Frontend Lead

KEY TASKS:

- Build admin/SystemSettings.jsx settings page
- Form to update SLA hours, auto-close timers, and broadcast message
- Add save action with confirmation alert

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build admin/SystemSettings.jsx.
Task: Create admin/SystemSettings.jsx screen.
1. Title: 'System Configuration & SLAs'.
2. Form fields:
   - Critical SLA (hours, default 24)
   - High SLA (hours, default 48)
   - Medium SLA (hours, default 72)
   - Citizen Verification Auto-Close window (hours, default 72)
   - Emergency Broadcast Advisory text.
3. Save button calling PATCH /api/v1/admin/settings/.
Generate admin/SystemSettings.jsx.
```

DAY 27 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Test Cross-Role Notification Flow across Citizen, Worker, and Officer
- Verify notifications arrive in real time for each status transition
- Document notification latency and delivery success rate

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Run cross-role notification test.
Task: Test and verify notification flow.
1. Open 3 browser windows:
   - Window 1: Citizen logged in
   - Window 2: Officer logged in
   - Window 3: Worker logged in
2. Execute workflow: Submit -> Assign -> Work Done -> Close.
3. Verify each role receives corresponding toast alert and bell increment.
4. Document test results in NOTIFICATION_INTEGRATION_REPORT.md.
Generate verification script and markdown report.
```

DAY 28 Anas Shaikh — Backend Lead

KEY TASKS:

- Phase 3 Complete Backend Verification: Run test suite across all 4 apps
- Verify zero SQL query warnings and clean database state
- Prepare database dump for staging deployment

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise Phase 3 Backend Verification.
Task: Run full backend test verification.
1. Run: python manage.py test accounts complaints dashboard notifications.
2. Verify all models, views, and tasks pass cleanly.
3. Verify test coverage is > 75%.
4. Generate database schema summary.
Generate test execution report.
```

DAY 28 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Verify PostGIS spatial queries under simulated load of 500 complaints
- Test ST_DWithin duplicate detection speed (< 20ms)
- Finalise dashboard KPI cache invalidation signals

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Benchmark Spatial Database Queries.
Task: Benchmark PostGIS queries and cache invalidation.
1. Seed 500 sample complaints across Nagpur coordinates.
2. Measure ST_DWithin query execution time with PostGIS GIST spatial index.
3. Connect Django post_save signal on Complaint to invalidate dashboard Redis cache.
Generate benchmark script and signal handlers.
```

DAY 28 Om Gadekar — Frontend Lead

KEY TASKS:

- Phase 3 Frontend Verification: Verify routing and protection for all 4 roles
- Test all 4 layouts: CitizenLayout, WorkerLayout, OfficerLayout, AdminLayout
- Verify zero broken links or unhandled errors

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise Phase 3 Frontend Routing.
Task: Verify all portal routes and layouts in src/App.jsx.
1. Check Citizen routes: /citizen, /citizen/complaints, /citizen/complaint/:id, /citizen/submit, /citizen/profile, /citizen/help.
2. Check Worker routes: /worker, /worker/completed, /worker/profile.
3. Check Officer routes: /officer, /officer/validation, /officer/workers, /officer/settings.
4. Check Admin routes: /admin, /admin/workers, /admin/complaints, /admin/reports, /admin/settings.
Generate routing verification checklist.
```

DAY 28 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Audit Notification Bell & Service Worker across desktop and mobile
- Verify offline push notification queueing and badge counter sync
- Finalise Phase 3 documentation update

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise Phase 3 Notification Audit.
Task: Audit notification bell and service worker.
1. Test background push receipt when browser tab is closed.
2. Test foreground toast display with 'View' button navigation.
3. Verify unread count badge decrements when notifications are read.
4. Document all notification handlers in NOTIFICATION_ARCHITECTURE.md.
Generate audit documentation.
```

Phase 4 -- Testing, Bugfix & Deployment — Days 29 -- 36

DAY 29 Anas Shaikh — Backend Lead

KEY TASKS:

- Run full Django test suite: python manage.py test
- Fix all failing tests across accounts, complaints, and notifications
- Add missing tests to reach 80%+ test coverage

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Run and fix all backend tests.
Task: Review test coverage and write missing tests.
1. Run existing tests: python manage.py test --verbosity=2.
2. List any failing tests and fix models, views, and serializers.
```

```
3. Write test cases for:
   - Worker assignment edge cases (ward mismatch rejection)
   - Citizen verification (rating bounds 1-5, reopen count increment)
   - FCM token registration
4. Verify coverage report: coverage run --source='.' manage.py test && coverage report -m.
Generate complete test suite additions.
```

DAY 29 — Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Write production-ready AI module with comprehensive error handling
- Integrate fine-tuned YOLO model with fallback simulation mode
- Benchmark AI pipeline execution under concurrent requests

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise production AI module.
Task: Harden ai_module code for deployment.
1. Add try-except guards in exif_checker, ela_checker, and yolo_validator.
2. Ensure pipeline never raises unhandled exceptions or crashes Celery worker.
3. If image format is invalid: return structured rejection with clear citizen message.
4. Add execution time logging for each pipeline stage.
Generate hardened AI pipeline code.
```

DAY 29 — Om Gadekar — Frontend Lead

KEY TASKS:

- Add smooth micro-animations, hover states, and transitions across all portals
- Audit mobile responsiveness on Android and iOS viewport dimensions
- Add global error boundary to catch and report React runtime errors

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Add UI micro-animations and Error Boundary.
Task: Polish React frontend design and error handling.
1. Add subtle scale/shadow micro-animations on button hovers and card presses.
2. Add ErrorBoundary component in src/components/ErrorBoundary.jsx:
   - Catches uncaught render errors.
   - Shows friendly civic fallback UI with 'Reload Page' button.
3. Audit all forms for keyboard navigation and screen-reader accessibility.
Generate ErrorBoundary.jsx and updated tailwind animations.
```

DAY 29 — Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Write comprehensive demo seed command: python manage.py seed_demo_accounts
- Create 1 Admin, 4 Dept. Officers, 16 Ward Workers (4x4), and 4 Citizens
- Seed 20 realistic complaints across all 4 departments and 4 wards

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build the Master Demo Seeder Command.
Task: Create accounts/management/commands/seed_demo_accounts.py.
1. Create Users:
   - Super Admin: 9900000000 / admin@123
   - 4 Officers: PWD (9900000001), SWM (9900000002), PHE (9900000003), Electrical (9900000004)
   - 16 Workers: 1 per (dept, ward) combination (e.g. NMC-PWD-W1: 9900001101 / worker@123)
   - 4 Citizens: 1 per ward (e.g. Citizen Ward 1: 9900002001 / citizen@123)
2. Seed 20 Complaints across Nagpur with realistic coordinates, descriptions, and statuses.
Generate complete seed_demo_accounts.py command.
```

DAY 30 Anas Shaikh — Backend Lead

KEY TASKS:

- Configure Celery task error handling and automatic retry policies
- Set max_retries=3 with exponential backoff for network/FCM tasks
- Add dead-letter task failure logging

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Add Celery Task Retry Policies.
Task: Update Celery tasks with retry decorators.
1. @shared_task(bind=True, max_retries=3, default_retry_delay=60) on send_push_notification.
2. @shared_task(bind=True, max_retries=2, default_retry_delay=30) on process_complaint_ai.
3. Log failed tasks to Celery task failure logger.
Generate updated tasks.py with resilient retry logic.
```

DAY 30 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Optimize YOLOv8 model inference speed using ONNX runtime / TensorRT
- Reduce memory footprint to fit within 512MB RAM cloud container limits
- Document inference latency benchmarks

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Optimize AI Model Memory Footprint.
Task: Optimize model weights for cloud deployment.
1. Export PyTorch model to ONNX: model.export(format='onnx', imgsz=320, half=True).
2. Measure RAM consumption during inference (< 250MB).
3. Benchmark inference time on CPU: ensure < 350ms per complaint image.
Generate optimization script and benchmarking report.
```

DAY 30 Om Gadekar — Frontend Lead

KEY TASKS:

- Audit Leaflet map performance and touch gestures on mobile devices
- Add custom map marker pins for 4 civic categories (Roads, Waste, Water, Light)
- Implement marker clustering for high-density complaint views

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Enhance Leaflet Map Components.
Task: Update Leaflet map wrappers in citizen and worker portals.
1. Custom SVG marker pins colored by category:
   - Blue for PWD, Green for SWM, Cyan for PHE, Yellow for Electrical.
2. Smooth pinch-to-zoom and pan on touch devices.
3. Auto-fit bounds to show all complaint markers in worker route view.
Generate updated map components.
```

DAY 30 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Conduct End-to-End User Journey Walkthrough testing
- Verify Citizen submit -> AI triage -> Officer assign -> Worker repair -> Citizen accept
- Record step-by-step test execution log

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Run End-to-End User Journey Test.
Task: Execute and document full complaint lifecycle test.
1. Step 1: Submit complaint via /citizen/submit with photo and GPS.
2. Step 2: Verify AI task validates complaint and routes to PWD.
3. Step 3: Login as PWD Officer -> assign Ward 2 worker.
4. Step 4: Login as Worker -> view task -> upload proof photo -> Mark Done.
5. Step 5: Login as Citizen -> verify resolution with 5-star rating -> status Closed.
Generate END_TO_END_TEST_REPORT.md.
```

DAY 31 Anas Shaikh — Backend Lead

KEY TASKS:

- Configure PostgreSQL connection pooling and transaction settings
- Tune CONN_MAX_AGE for persistent database connections
- Benchmark database throughput under concurrent queries

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Configure DB Connection Pooling.
Task: Update settings/production.py and database configuration.
1. Configure dj_database_url with conn_max_age=600 and ssl_require=True.
2. Add django.db.transaction.atomic() on complaint submission and worker assignment.
3. Test database recovery under connection drop simulation.
Generate updated database settings.
```

DAY 31 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Test duplicate detection spatial edge cases near ward boundaries
- Verify 100m radius threshold accuracy using PostGIS ST_Distance
- Document boundary handling in spatial architecture notes

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Test PostGIS Boundary Edge Cases.
Task: Write spatial edge-case test suite in ai_module/tests.py.
1. Test complaints 95m apart -> verify flagged as duplicate.
2. Test complaints 105m apart -> verify treated as separate complaints.
3. Test complaints across ward boundary lines.
Generate spatial unit test cases.
```

DAY 31 Om Gadekar — Frontend Lead

KEY TASKS:

- Optimize React build bundle size using dynamic imports and code splitting
- Lazy-load Recharts, Leaflet, and heavy dashboard views via React.lazy
- Verify initial JavaScript bundle size is under 200KB gzip

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Optimize Frontend Bundle Size.
Task: Configure code splitting in src/App.jsx.
1. Replace static imports of heavy pages with React.lazy and React.Suspense:
   - const CrossDeptDashboard = React.lazy(() => import('./pages/admin/CrossDeptDashboard'));
   - const OfficerDashboard = React.lazy(() => import('./pages/officer/Dashboard'));
   - const MyComplaints = React.lazy(() => import('./pages/citizen/MyComplaints'));
2. Add sleek Suspense loading spinner.
3. Verify npm run build output reports optimized chunks.
Generate updated App.jsx with code splitting.
```

DAY 31 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Test Firebase Cloud Messaging push notifications in HTTPS production environment
- Verify delivery on Android Chrome, Windows Chrome/Edge, and macOS Safari
- Document notification reception latency (< 3 seconds)

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Verify FCM Push on Production HTTPS.
Task: Test live web push delivery.
1. Deploy frontend to HTTPS hosting (Render / Vercel).
2. Register FCM push token and verify token syncs to Django backend.
3. Dispatch background test notification via management command.
4. Verify OS system tray banner appears within 3 seconds.
Generate PRODUCTION_NOTIFICATION_AUDIT.md.
```

DAY 32 Anas Shaikh — Backend Lead

KEY TASKS:

- Create production Docker Compose configuration: docker-compose.prod.yml
- Configure Gunicorn WSGI server with 3 worker processes
- Add WhiteNoise middleware for compressed static asset serving

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Create Production Docker Compose.
Task: Create docker-compose.prod.yml and configure Gunicorn.
1. docker-compose.prod.yml with backend, celery, postgres, redis services.
2. Gunicorn command: gunicorn resolveIT.wsgi:application --bind 0.0.0.0:8000 --workers 3 --threads 2.
3. WhiteNoise settings in base.py:
   - STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
Generate docker-compose.prod.yml and static settings.
```

DAY 32 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Configure Redis Caching for Dashboard KPI and Ward Stats endpoints
- Set cache TTL = 300 seconds with automated signal-based cache invalidation
- Benchmark API response latency reduction (from 120ms to 8ms)

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Implement Redis Dashboard Caching.
Task: Add Redis caching to dashboard/views.py.
1. Cache GET /api/v1/dashboard/kpi/ response in Redis: cache.get_or_set('dashboard_kpi_all', ...).
2. Cache GET /api/v1/dashboard/ward-stats/ response in Redis.
3. Django signal invalidates cache when Complaint status changes.
Generate caching implementation in dashboard views.
```

DAY 32 Om Gadekar — Frontend Lead

KEY TASKS:

- Configure Vite production build settings and asset hashing
- Create public/_redirects file for Single Page Application routing on Render
- Verify clean page refresh on deep URLs (/citizen/complaint/:id)

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Configure SPA Production Routing.
Task: Create public/_redirects and configure vite.config.js.
1. Create public/_redirects with: /* /index.html 200
2. Update vite.config.js with build chunking and asset minification.
3. Test direct URL refresh on nested routes (/admin/workers, /citizen/complaint/123).
Generate _redirects file and updated vite.config.js.
```

DAY 32 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Harden service worker caching strategy in firebase-messaging-sw.js
- Cache app shell and notification icons for offline accessibility
- Add service worker update event listener in React

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Harden Service Worker Caching.
Task: Enhance public/firebase-messaging-sw.js.
1. Cache static assets: /icons/janmitra-logo-192.png, /icons/badge-72.png.
2. Background sync fallback if device is temporarily offline when push arrives.
3. Handle new service worker update without forcing page reload.
Generate updated firebase-messaging-sw.js.
```

DAY 33 Anas Shaikh — Backend Lead

KEY TASKS:

- Create Render Cloud Deployment Configuration: render.yaml blueprint
- Configure Web Service, Celery Background Worker, and Managed PostgreSQL
- Document environment variables setup guide

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Create Render Blueprint Configuration.
Task: Create render.yaml infrastructure blueprint.
1. Web service: resolveit-backend (Python 3.11, Gunicorn, port 8000).
2. Worker service: resolveit-celery (Celery worker, 2 concurrency).
3. Database: resolveit-postgres (PostgreSQL 15 with PostGIS extension).
4. Environment variables mapping: SECRET_KEY, DATABASE_URL, REDIS_URL, FRONTEND_URL.
Generate render.yaml and RENDER_DEPLOYMENT_GUIDE.md.
```

DAY 33 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Configure Celery Worker resource management and concurrency
- Set worker memory leak guard: --max-tasks-per-child=100
- Verify worker health check endpoint

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Configure Celery Worker Health & Limits.
Task: Configure Celery deployment parameters.
1. Celery startup command with concurrency limits:
   celery -A resolveIT worker -l INFO --concurrency=2 --max-tasks-per-child=100.
2. Add Celery ping verification script check_celery_health.py.
Generate Celery deployment settings.
```

DAY 33 Om Gadekar — Frontend Lead

KEY TASKS:

- Deploy React Frontend to Render Static Site
- Configure environment variables: VITE_API_URL pointing to production backend
- Verify CORS headers allow requests from frontend domain

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Deploy Frontend to Render.
Task: Set up Render Static Site deployment.
1. Connect GitHub repository to Render Static Site.
2. Build command: npm install && npm run build
3. Publish directory: dist
4. Environment variable: VITE_API_URL = https://resolveit-backend.onrender.com
Generate FRONTEND_DEPLOYMENT_CHECKLIST.md.
```

DAY 33 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Write notification system tests and a debugging guide: notification_test_guide.md
- Document 4 core tests: Complaint Submitted, Validated, Worker Assigned, Background OS alert
- Document common FCM issues and step-by-step browser fixes

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Write notification system tests and debugging guide.
Task: Create notification_test_guide.md.
1. Step-by-step test guide:
   - Test 1: Complaint Submitted notification
   - Test 2: Complaint Validated notification
   - Test 3: Worker Assigned notification
   - Test 4: Background notification (app closed)
2. Common FCM issues and fixes:
   - VAPID key missing
```

```
   - Service worker not registered
   - Notification permission blocked (browser reset guide)
   - FCM token not saved (PATCH /auth/fcm-token/)
Generate notification_test_guide.md completely.
```

DAY 34 Anas Shaikh — Backend Lead

KEY TASKS:

- Create Backend Health Check API endpoint: GET /api/v1/health/
- Verify database connectivity, Redis ping, and Celery worker responsiveness
- Return structured JSON health status for uptime monitoring

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build System Health Check API.
Task: Create HealthCheckView in accounts/views.py.
1. Endpoint: GET /api/v1/health/ (AllowAny)
2. Checks:
   - Database: User.objects.count() -> ok
   - Redis: redis_client.ping() -> ok
   - Celery: celery_app.control.ping() -> ok
3. Response: { status: 'healthy', database: 'connected', redis: 'connected', timestamp: '...' }
Generate health check endpoint and URL route.
```

DAY 34 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Configure UptimeRobot free ping monitor to keep Render web service awake
- Set 10-minute HTTP ping to /api/v1/health/ endpoint
- Eliminate Render free tier cold-start latency

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Setup Uptime Monitor to prevent cold starts.
Task: Document and configure UptimeRobot ping.
1. Add UptimeRobot HTTP monitor pointing to https://resolveit-backend.onrender.com/api/v1/health/.
2. Set interval: Every 10 minutes (prevents Render 15-minute inactivity spin-down).
3. Document setup in UPTIME_CONFIG.md.
Generate uptime configuration guide.
```

DAY 34 Om Gadekar — Frontend Lead

KEY TASKS:

- Build Frontend /health diagnostics page
- Display current API baseURL, build timestamp, and network connectivity status
- Useful for verifying production environment configuration

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Build Frontend Health Diagnostics Page.
Task: Create src/pages/HealthCheck.jsx.
1. Route: /health
2. Displays:
   - API Base URL: import.meta.env.VITE_API_URL
   - Backend Ping: Live status check calling /api/v1/health/
   - Browser Service Worker Status: Registered / Active
   - LocalStorage Auth Status: Logged in / Guest
Generate HealthCheck.jsx component.
```

DAY 34 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Write Django management command: send_test_notification.py
- Usage: python manage.py send_test_notification --user-id=<UUID>
- Output full diagnostic terminal trace of token and delivery status

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Write send_test_notification command.
Task: Create notifications/management/commands/send_test_notification.py.
1. Accepts: --user-id=<UUID>, optional --title, --body, --complaint-id.
2. Validates user exists and checks user.fcm_token.
3. Tests Firebase Admin initialization (with safe simulation fallback).
4. Dispatches push and stores in-app Notification database record.
Generate complete send_test_notification.py command.
```

DAY 35 Anas Shaikh — Backend Lead

KEY TASKS:

- Execute database migration on production Render cloud instance
- Run python manage.py migrate and verify PostGIS spatial tables
- Execute seed_demo_accounts command on production shell

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Run Production Database Migrations.
Task: Execute cloud deployment migration checklist.
1. Connect to Render shell:
   - python manage.py migrate --noinput
   - python manage.py seed_demo_accounts
2. Verify all 4 categories, 4 wards, and demo accounts are created.
3. Verify superadmin user access.
Generate PRODUCTION_MIGRATION_LOG.md.
```

DAY 35 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Verify AI validation and PostGIS duplicate detection on production database
- Submit test complaint on production URL and verify Celery processes task
- Verify dashboard KPIs calculate correctly with seeded data

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Verify AI Module on Production Cloud.
Task: Test production AI pipeline execution.
1. Trigger AI validation on production Celery worker.
2. Check Celery logs: verify 'process_complaint_ai succeeded'.
3. Verify duplicate detection query executes correctly with PostGIS extension.
Generate AI_PRODUCTION_VERIFICATION.md.
```

DAY 35 Om Gadekar — Frontend Lead

KEY TASKS:

- Perform smoke tests across all 4 production portals on live Render URL
- Test Citizen submission, Worker task completion, Officer assignment, Admin dashboard
- Verify zero console errors in browser DevTools

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Run Production Frontend Smoke Tests.
Task: Execute smoke testing on live website.
1. Test citizen login and home page loading on mobile browser.
2. Test submitting complaint with real camera photo on mobile phone.
3. Test officer dashboard Recharts rendering and ward heatmap display.
4. Verify all API requests return 200 OK without CORS errors.
Generate FRONTEND_SMOKE_TEST_REPORT.md.
```

DAY 35 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Write Production Testing Checklist: PRODUCTION_TEST_CHECKLIST.md
- Document testing across Phase 1 Health, Phase 2 Auth, Phase 3 Lifecycle, Phase 4 Dashboard
- List all pre-configured demo account credentials for viva presentation

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Write production testing checklist.
Task: Create PRODUCTION_TEST_CHECKLIST.md.
1. Health check & database verification items.
2. Auth flow test checklist for all 4 user roles.
3. Complaint lifecycle step-by-step verification checklist.
4. Complete Demo Accounts Reference Table:
   - Admin: 9900000000 / admin@123
   - 4 Officers: PWD (9900000001), SWM (9900000002), PHE (9900000003), Elec (9900000004)
   - 16 Ward Workers (1 per dept+ward combination)
   - 4 Ward Citizens
Generate PRODUCTION_TEST_CHECKLIST.md completely.
```

DAY 36 Anas Shaikh — Backend Lead

KEY TASKS:

- Final Security Audit: Verify environment variable masking and secret key isolation
- Disable DEBUG mode permanently in production settings
- Configure HTTP Security headers (HSTS, X-Content-Type-Options, X-Frame-Options)

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Configure HTTP Security Headers.
Task: Finalise security settings in settings/production.py.
1. SECURE_SSL_REDIRECT = True
2. SECURE_HSTS_SECONDS = 31536000
3. SECURE_HSTS_INCLUDE_SUBDOMAINS = True
4. SECURE_CONTENT_TYPE_NOSNIFF = True
5. X_FRAME_OPTIONS = 'DENY'
6. Verify no sensitive credentials appear in git repository.
Generate security settings additions.
```

DAY 36 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Finalise AI Model Evaluation Metrics and Confusion Matrix documentation
- Document precision, recall, and F1-score across all 4 civic issue categories
- Prepare algorithm justification notes for college viva defense

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Document AI Model Evaluation Metrics.
Task: Create AI_MODEL_EVALUATION.md.
1. Confusion matrix across: Pothole, Garbage Dump, Water Leakage, Streetlight Damage.
2. Precision (91.4%), Recall (88.7%), F1-Score (90.0%).
3. AI Fake Detection accuracy metrics (94.2% on synthetic generated images).
4. Prepare Viva Q&A defense answers for external examiners.
Generate AI evaluation report.
```

DAY 36 Om Gadekar — Frontend Lead

KEY TASKS:

- Add PWA Web App Manifest (manifest.json) for mobile home-screen installability
- Configure app icons (192x192, 512x512) and theme-color (#0d2a53)
- Enable citizens to add JANMITRA as native-feeling phone app

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Add PWA Manifest to Frontend.
Task: Create public/manifest.json and update index.html.
1. manifest.json:
   - name: 'JANMITRA - Civic Issue Management'
   - short_name: 'JANMITRA'
   - theme_color: '#0d2a53', background_color: '#f8fafc'
   - display: 'standalone', orientation: 'portrait'
   - icons: 192x192 and 512x512 PNG icons.
2. Add Apple touch icon and meta viewport in index.html.
Generate manifest.json and updated index.html.
```

DAY 36 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Finalise notification permission UX: show gentle unblock prompt if blocked
- Add foreground toast with 'View' button navigation in firebase.js
- Audit notification settings across all 4 portals

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise Notification UX.
Task: Update src/firebase.js with gentle prompt and styled toast.
1. If Notification.permission === 'denied': show gentle floating banner explaining how to click browser padlock icon.
2. onForegroundMessage: render toast with Title, Body, and 'View' CTA button.
3. Test end-to-end receipt on active and inactive browser states.
Generate updated firebase.js.
```

Phase 5 -- Demo Prep, Documentation & Viva — Days 37 -- 40

DAY 37 Anas Shaikh — Backend Lead

KEY TASKS:

- Create Master GitHub Repository README.md with complete setup instructions
- Document architecture diagram, technology stack, and local installation steps
- Add badges for Python, Django, PostgreSQL, PostGIS, Docker, and Render

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Create Master GitHub README.md.
Task: Write comprehensive README.md for root repository.
1. Project Overview & Nagpur Municipal Corporation problem statement.
2. Key Architectural Features (4 Categories, 4 Wards, AI validation, PostGIS 100m, FCM).
3. System Architecture Mermaid Diagram.
4. Quickstart Guide (Local Docker Compose & Cloud Deployment).
5. API Endpoints Table.
6. Team ResolveIT contribution credits.
Generate README.md completely.
```

DAY 37 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Create Technical Documentation for AI Validation & Priority Scoring Module
- Explain Layer 1 Fake Detection (EXIF + ELA + HuggingFace) and Layer 2 YOLOv8
- Explain PostGIS spatial duplicate detection and formula calculation

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Create AI Technical Documentation.
Task: Create AI_ARCHITECTURE_DOCUMENTATION.md.
1. Detailed algorithm flowchart for photo validation.
2. Mathematical breakdown of Priority Score formula.
3. PostGIS ST_DWithin spatial query explanation with diagram.
4. Viva questions and answers for AI module defense.
Generate AI architecture documentation.
```

DAY 37 Om Gadekar — Frontend Lead

KEY TASKS:

- Capture presentation-quality screenshots across all 4 portal views
- Include Citizen Home, Submit Form with Map, Stepper, Worker Tasks, Officer Dashboard, Admin Heatmap
- Format screenshots with professional device mockups for project report

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Capture UI Presentation Screenshots.
Task: Create UI_SCREENSHOTS_CATALOG.md.
1. List of 12 high-resolution screenshots to capture.
2. Descriptions for each screen:
   - Screen 1: Multi-Role Login Page
   - Screen 2: Citizen Home with Quick Stats
   - Screen 3: Issue Submission with Leaflet GPS Map
   - Screen 4: 6-Step Lifecycle Stepper
   - Screen 5: Worker Task List with Priority Badges
   - Screen 6: Worker Proof-of-Work Photo Upload
   - Screen 7: Citizen Verification & 5-Star Rating
   - Screen 8: Department Officer Dashboard
   - Screen 9: Super Admin Cross-Dept Overview & Heatmap
   - Screen 10: Worker Management Panel
3. Annotate key civic governance UI features.
Generate UI screenshots catalog.
```

DAY 37 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Write live demonstration script for push notifications in project presentation
- Prepare dual-screen demonstration: submit on phone -> instant alert on laptop
- Document fallback demo steps in case college Wi-Fi restricts WebSockets/FCM

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Create Push Notification Demo Guide.
Task: Create NOTIFICATION_DEMO_GUIDE.md.
1. Dual-screen live demo procedure for project viva:
   - Screen A (Phone): Citizen submits civic issue.
   - Screen B (Laptop): Officer receives real-time validation notification.
   - Officer assigns worker -> Phone receives 'New Task Assigned'.
2. Offline / Simulation backup instructions using send_test_notification command.
Generate demo guide.
```

DAY 38 Anas Shaikh — Backend Lead

KEY TASKS:

- Export complete Postman Collection and OpenAPI schema for all 24 API endpoints
- Include sample request bodies, query parameters, and authenticated headers
- Save resolveit_janmitra_api.postman_collection.json

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Export Postman Collection.
Task: Create Postman API Collection JSON file.
1. Group requests by folder: Auth, Complaints, Worker, Dashboard, Admin.
2. Set environment variables: {{base_url}}, {{access_token}}.
3. Include pre-request script to auto-inject Bearer JWT token.
Generate postman collection export file.
```

DAY 38 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Document PostGIS Spatial Performance & Nagpur Municipal Ward Boundaries
- Explain GeoJSON boundary polygons for Wards 1 to 4
- Document spatial indexing performance benchmarks

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Document Spatial Database Architecture.
Task: Create SPATIAL_DATABASE_REPORT.md.
1. Detailed explanation of PostGIS geography vs geometry types.
2. GIST spatial index structure on complaint location field.
3. GeoJSON boundaries definition for Nagpur Central, North, East, West wards.
Generate spatial architecture report.
```

DAY 38 Om Gadekar — Frontend Lead

KEY TASKS:

- Create final Project Presentation Slide Deck: JANMITRA_Seminar_PPT.pptx
- Include Problem Statement, Solution, Architecture, 4 Portals, Demo Video link
- Include Team Contribution slides for all 4 members

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Prepare Seminar Presentation PPT.
Task: Create seminar presentation outline and slide generator.
1. Slide 1: Title Slide (JANMITRA, Team ResolveIT, NMC Nagpur).
2. Slide 2: The Civic Crisis in Indian Cities (Nagpur context).
3. Slide 3: JANMITRA Solution Architecture.
4. Slide 4: 4 Civic Categories, 4 Wards, 4 Departments.
5. Slide 5: AI Validation Pipeline (EXIF, ELA, YOLOv8).
6. Slide 6: Spatial Duplicate Detection (PostGIS 100m).
7. Slide 7: Complete Complaint Lifecycle Stepper.
8. Slide 8: Department Officer Dashboard & Heatmap.
9. Slide 9: Field Worker Portal & Proof of Work.
10. Slide 10: Live Demonstration & Results.
11. Slide 11: Technology Stack & Cloud Deployment.
12. Slide 12: Team Contributions & Conclusion.
Generate presentation outline and python-pptx script.
```

DAY 38 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Format Project Report Executive Summary and Chapter 1: Introduction
- Document project background, scope, and objectives for B.Tech thesis
- Format academic citations and municipal governance references

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Write Report Chapter 1: Introduction.
Task: Create REPORT_CHAPTER_1_INTRODUCTION.md.
1. Background of Urban Local Bodies (ULB) civic complaint handling.
2. Problem Statement: Absence of automated tracking and verification in NMC.
3. Project Objectives: Instant reporting, AI validation, worker routing, citizen verification.
4. Scope and Limitations (4 Categories, 4 Wards constraint).
5. Organization of the Project Report.
Generate complete academic chapter text.
```

DAY 39 Anas Shaikh — Backend Lead

KEY TASKS:

- Document Database Schema and Entity-Relationship (ER) Diagram
- Include all 8 core tables: users, categories, wards, departments, complaints, complaint_images, status_history, notifications
- Document foreign key constraints and cascade rules

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Document Database ER Diagram.
Task: Create DATABASE_SCHEMA_ERD.md.
1. Mermaid ER diagram showing all 8 tables and relationships.
2. Data dictionary table detailing data types, nullability, and default values.
3. Indexing strategy explanation for high-performance municipal queries.
Generate complete database schema documentation.
```

DAY 39 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Prepare Viva Technical Defense Guide for AI & Data Science External Examiners
- Formulate answers for: Why YOLOv8 over ResNet? Why 100m duplicate radius? How does ELA work?
- Document AI limitations and future edge-AI improvements

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Create Viva Technical Defense Guide.
Task: Create VIVA_DEFENSE_GUIDE_AI.md.
1. 10 most anticipated external examiner questions with expert technical answers.
2. Mathematical justification of Priority Engine factor weights.
3. Explanation of Error Level Analysis (JPEG compression artifacts).
4. Spatial query complexity analysis (R-Tree GIST index O(log N)).
Generate comprehensive viva defense guide.
```

DAY 39 Om Gadekar — Frontend Lead

KEY TASKS:

- Write Live Demonstration Script with exact timings (10-minute presentation)
- Script roles: Anas presents Architecture, Anurag presents AI, Om presents Dashboard, Faizan presents Citizen & Push
- Include emergency fallback demonstration recordings

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Write 10-Minute Presentation Script.
Task: Create LIVE_DEMO_TIMED_SCRIPT.md.
1. Minute 0-2: Introduction & Problem Context (All).
2. Minute 2-4: Citizen Reporting & Live Map Location (Faizan).
3. Minute 4-6: AI Validation & PostGIS Duplicate Engine (Anurag).
4. Minute 6-8: Officer Dashboard, Heatmap & Worker Roster (Om).
5. Minute 8-9: Backend Infrastructure & Architecture (Anas).
6. Minute 9-10: Conclusion & Examiner Q&A.
```

```
Generate timed presentation script.
```

DAY 39 Faizan Rayeen — Citizen + Notifications

KEY TASKS:

- Create Demo Video Recording Plan and Script for complete complaint lifecycle
- Record 3-minute high-definition demonstration walkthrough
- Include voiceover script and key feature highlight callouts

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Create Demo Video Walkthrough Script.
Task: Create DEMO_VIDEO_SCRIPT.md.
1. Scene-by-scene script for 3-minute demonstration video.
2. Callouts: AI Fake Check, 100m Duplicate Flag, Push Notification Toast, 6-Step Stepper, Proof of Work.
3. Subtitle text and audio voiceover guide.
Generate video recording guide.
```

DAY 40 Anas Shaikh — Backend Lead

KEY TASKS:

- Final Production Release: Tag Git version v1.0.0
- Clean up debug print statements, console logs, and temporary test fixtures
- Sanitize all environment files and generate final production release archive

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Final Production Release & Cleanup.
Task: Prepare final v1.0.0 release archive.
1. Remove all debug print statements across Django codebase.
2. Verify .env is in .gitignore and .env.example contains no real passwords.
3. Tag git release: git tag -a v1.0.0 -m 'JANMITRA Final Production Release'.
4. Generate release notes summarizing all completed modules.
Generate final release documentation.
```

DAY 40 Anurag Deshmukh — AI + Dashboard

KEY TASKS:

- Package trained AI model weights and pipeline assets
- Backup weights/janmitra_civic_yolov8n.pt to cloud repository
- Finalise AI Module section of the B.Tech project technical report

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Finalise AI Technical Report Section.
Task: Create REPORT_CHAPTER_AI_MODULE.md.
1. Complete formal academic writeup of the AI validation pipeline.
2. Include precision/recall tables, confusion matrix figures, and training curves.
3. Document municipal deployment feasibility and edge inferencing.
Generate final academic chapter.
```

DAY 40 Om Gadekar — Frontend Lead

KEY TASKS:

- Final Frontend Build: Generate production distribution bundle
- Audit zero console warnings, zero broken images, and smooth 60fps animations
- Package all UI design assets, component documentation, and user guides

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Final Frontend Build & UI Documentation.
Task: Finalise frontend production distribution.
1. Run final production build: npm run build.
2. Verify dist/ folder contains optimized, minified assets.
3. Package UI component documentation in UI_COMPONENT_GUIDE.md.
4. Verify PWA install banner triggers correctly on mobile devices.
Generate final frontend handover documentation.
```

DAY 40 Faizan Rayeen Citizen + Notifications

KEY TASKS:

- Compile Executive Summary and Complete Project Handover Package
- Verify all 11 PDF project documents are generated, verified, and backed up
- Organize submission drive with code, database seeds, presentation, and report

>> ANTIGRAVITY PROMPT (copy-paste into IDE prompt bar)

```
You are helping me build JANMITRA. Final Project Submission Package.
Task: Compile final handover package and executive summary.
1. Executive Summary document covering all 40 days achievements.
2. Verify all 11 documentation PDFs are present in JANMITRA_Final_v3:
   - 01_PRD, 02_SRS, 03_Architecture, 04_UIUX, 05_Development, 06_Team,
   - 07_Dept_Dashboard, 08_Worker_Management, 09_TechStack,
   - 10_Project_Breakdown, 11_Day_by_Day_Prompts (Complete 40 Days).
3. Generate project completion sign-off document for college submission.
Generate final executive summary and completion package.
```